编译的复习

编译的复习
- 前言
- 第二章：词法 正则表达式 转换
- 第三章：自底向上重点，LALR不考
- 第四章：写SDD并执行，结合生成
- 第五章：到哪步做什么事情
- 第六章：3地址码，和第七章中间表示一起
- 第七章：3地址码，控制流图，有向无环图优化等
- 第八章：中期到达定义，活跃变量；后期寄存器分配
- 结尾

前言

先看PPT，只要看例题，不会再学，把不会的记录。

之后刷有答案的作业题。

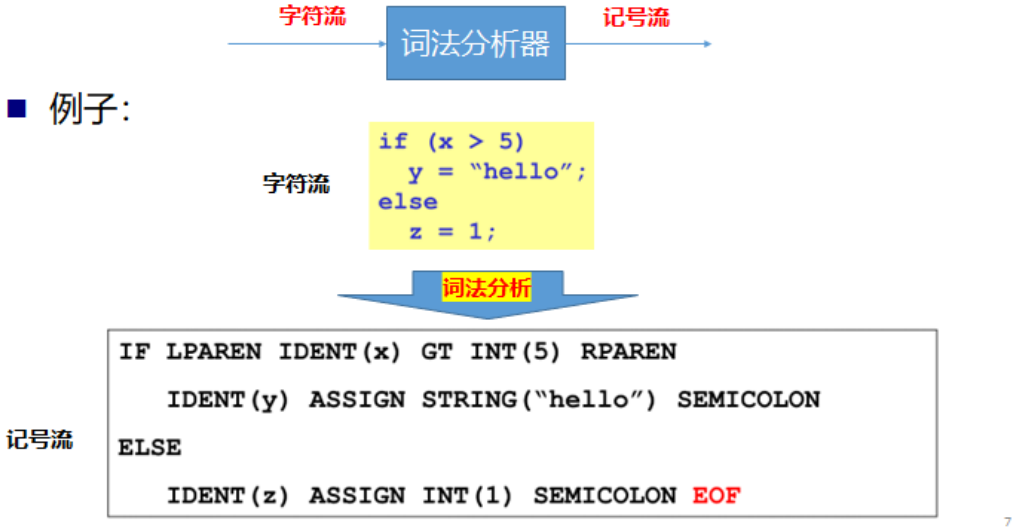
下面的知识点都要明白。

第二章：词法 正则表达式 转换
第三章：自底向上重点，LALR不考
第四章：写SDD并执行，结合生成
第五章：到哪步做什么事情
第六章：3地址码，和第七章中间表示一起
第七章：3地址码，控制流图，有向无环图优化等
第八章：中期到达定义，活跃变量；后期寄存器分配
第九章：非嵌套语言的运行时管理

第二章：词法 正则表达式 转换

- 词法分析器

词法分析器的任务

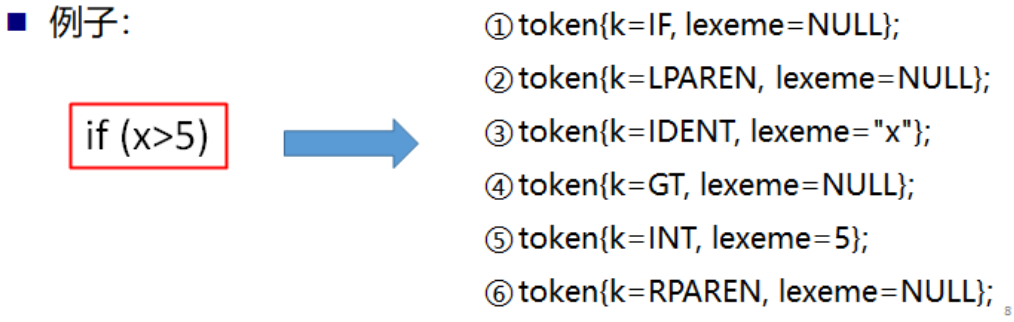


在做实验的时候已经完全知道了，就是写正则表达式匹配，之后做动作出词法。

- 记号的例子

记号的数据结构定义

```
enum kind {IF, LPAREN, IDENT, INT, GT...};
struct token{
        enum kind k;
        char *lexeme;
};
```



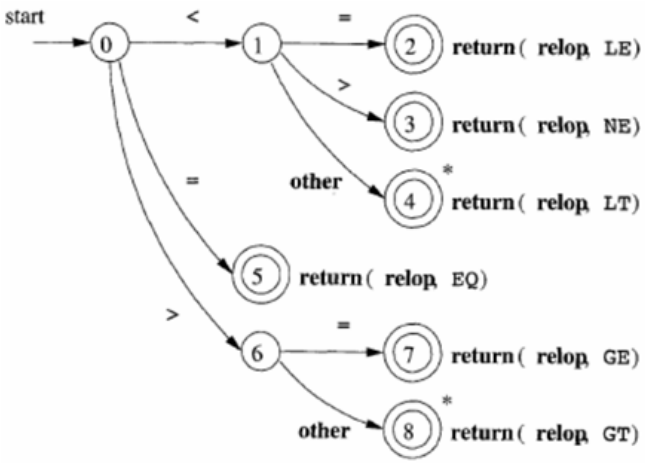
实现的数据结构例如上，类型+值。

- DFA例子

转移图

- 例子：

六种关系运算符：
<=
<>
<
=
>=
>



14

DFA匹配的例子，会写。

- 对关键字的处理

标识符和关键字

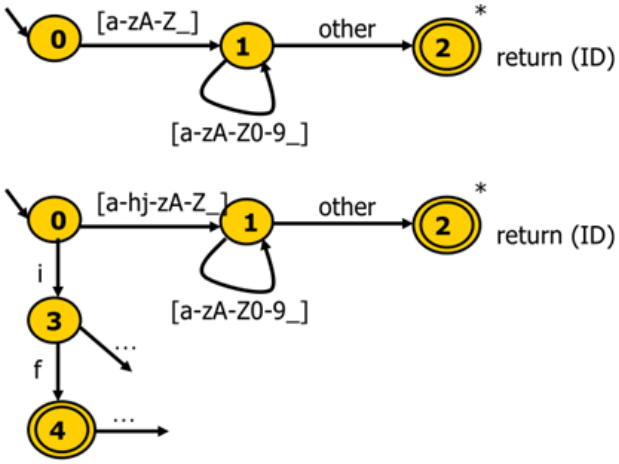
- 很多语言中的标识符和关键字有交集
  - 从词法分析的角度看，关键字是标识符的一部分
- 以C语言为例：
  - 标识符：以字母或下划线开头，后跟零个或多个字母、下划线、或数字
  - 关键字：**if, while, else, …**
- 两种关键字识别方法
  - 切分转移图
  - 关键字表算法

17

标识符和关键字，要把关键字给区分出来。

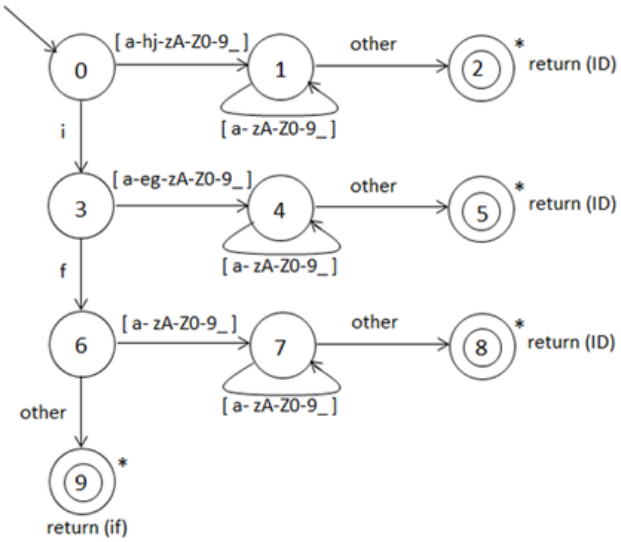
- 对关键字的处理1

切分转移图（以if为例）



18

切分转移图（以if为例）



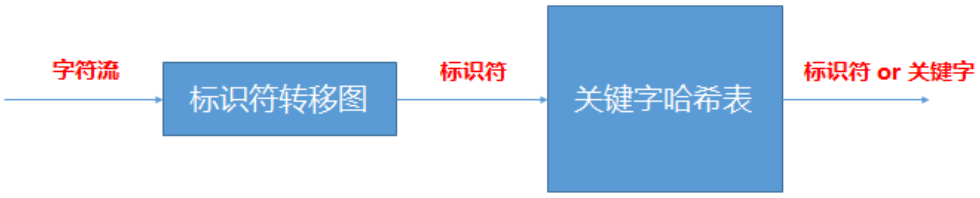
19

切分转移图，为关键字设置分支，切开了标识符的字符范围。

- 对关键字的处理2

关键字表算法

- 对给定语言中所有的关键字，构造关键字构成的哈希表H
- 对所有的标识符和关键字，先统一按标识符的转移图进行识别
- 识别完成后，进一步查表H看是否是关键字
- 通过合理的构造哈希表H（完美哈希），可以O（1）时间完成



先识别标识符，再看表选出子集关键字

- 正则表达式表示

例子：关键字

e -> ε
| c∈∑
| e | e
| e e
| e*

- C语言中的关键字，例如if，while等
  - 如何用**正则表达式**表示?

① **if**
② **while**

例子：标识符

e -> ε
| c∈∑
| e | e
| e e
| e*

- C语言中的标识符：以字母或下划线开头，后跟零个或多个字母、数字或下划线。
  - 如何用**正则表达式**表示?

① **(字母或下划线)(零个或多个字母、数字或下划线)**
② **(a|b|...|z|A|B|...|Z|_)(零个或多个字母、数字或下划线)**
③ **(a|b|...|z|A|B|...|Z|_)((字母、数字或下划线)*)**
④ **(a|b|...|z|A|B|...|Z|_)((a|b|...|z|A|B|...|Z|0|1|...|9|_)*)**

把意思逐渐实现为规范形式的1~4，是标识符的正则表达式实现法

例子：C语言中的无符号整数

e -> ε
| c∈∑
| e | e
| e e
| e*

- （十进制整型数）规则：或者是0；或者是以1到9开头，后跟零个或多个0到9
  - 如何用**正则表达式**表示?

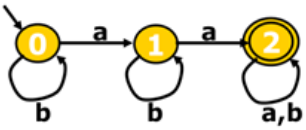
① **0|((1|2|...|9)(0|1|2|...|9)*)**

无符号整数的正则表达式方法，没有符号，注意0开头只可能是0，不然要非0开头。

- 自动机

自动机例子

M=(∑, S, q0, F, δ)



- 字母表
  - ∑={a, b}
- 状态集
  - S={0, 1, 2}
- 初始状态
  - q0=0
- 终结状态集
  - F={2}

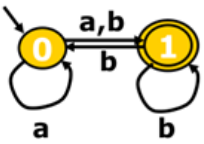
- 转移函数
  - { (0,a) -> 1, (0,b) -> 0,
    (1,a) -> 2, (1,b) -> 1,
    (2,a) -> 2, (2,b) -> 2 }

34

DFA

自动机例子

M=(∑, S, q0, F, δ)



- 字母表
  - ∑={a, b}
- 状态集
  - S={0, 1}
- 初始状态
  - q0=0
- 终结状态集
  - F={1}

- 转移函数
  - { (0,a) -> {0, 1},
    (0,b) -> {1},
    (1,b) -> {0, 1} }

35

NFA

这门课自动机分为

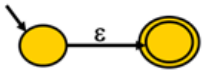
$$\{\text{字母表}\Sigma, \text{状态}S, \text{转移函数}\delta, \text{初始状态}q_0, \text{终止状态集合}F\}$$

其实和之前的区别就是换了个位置，不用太注意。到时候把每一个分别写出来就可以避免这个问题。

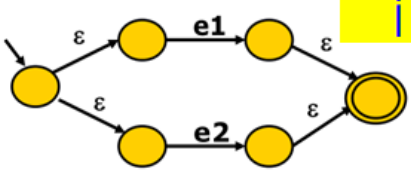
- 任何一个RE化为NFA

Thompson算法

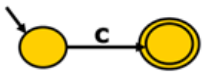
e -> ε
| c∈∑
| e e
| e | e
| e*



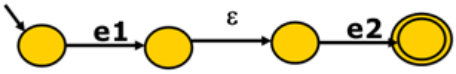
e -> ε



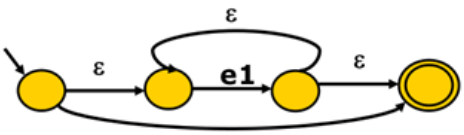
e -> e1 | e2



e -> c



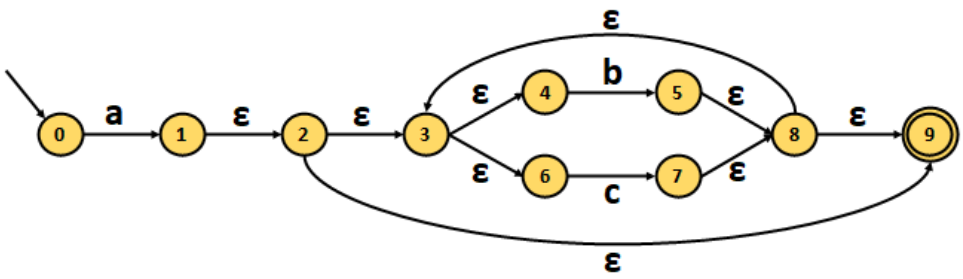
e -> e1 e2

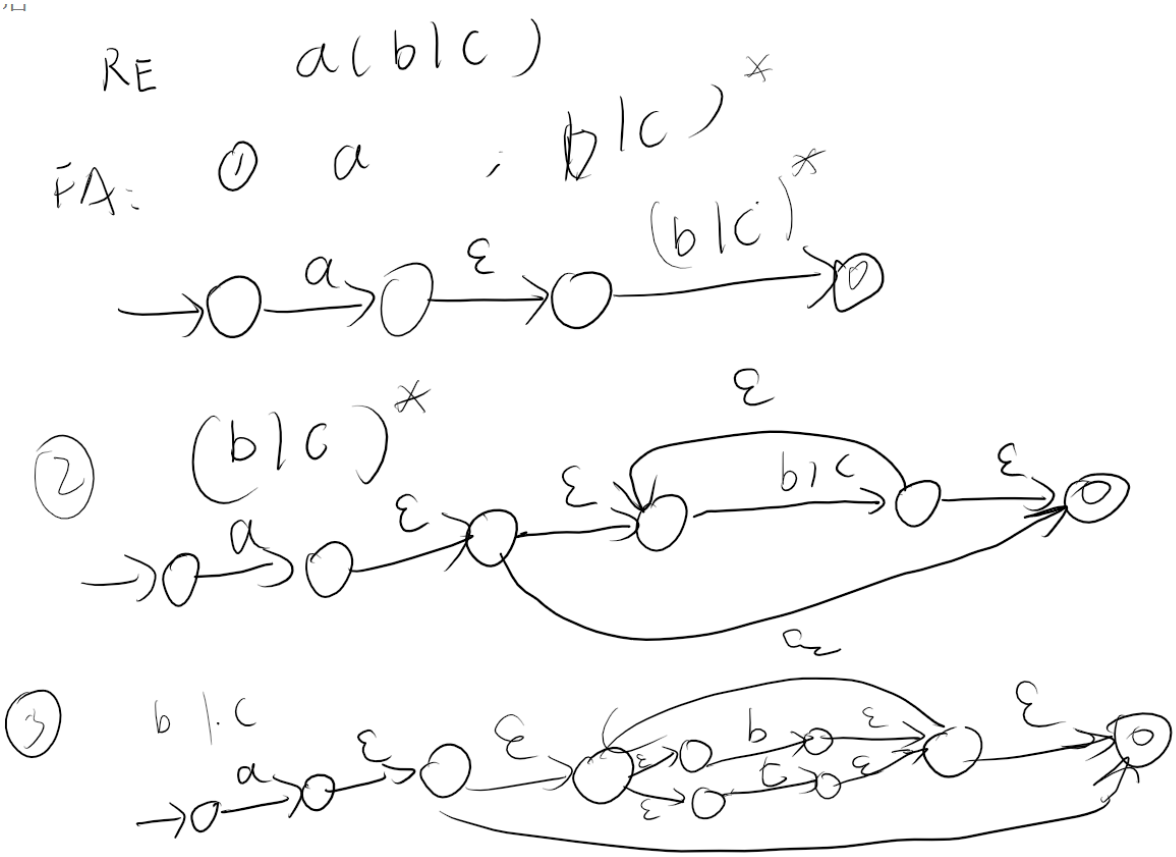


e -> e1*

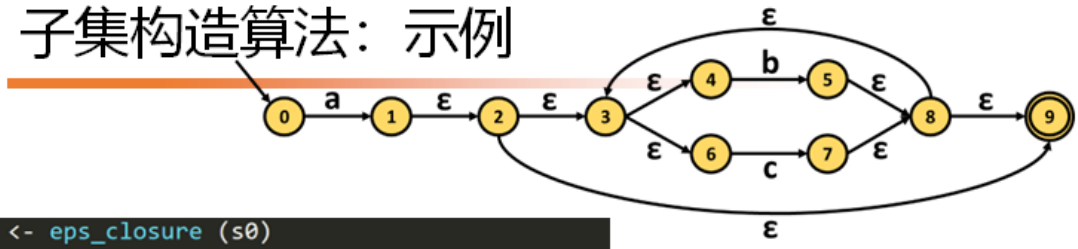
示例

a (b | c)*





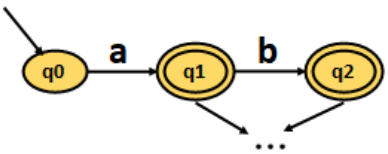
- NFA化为DFA



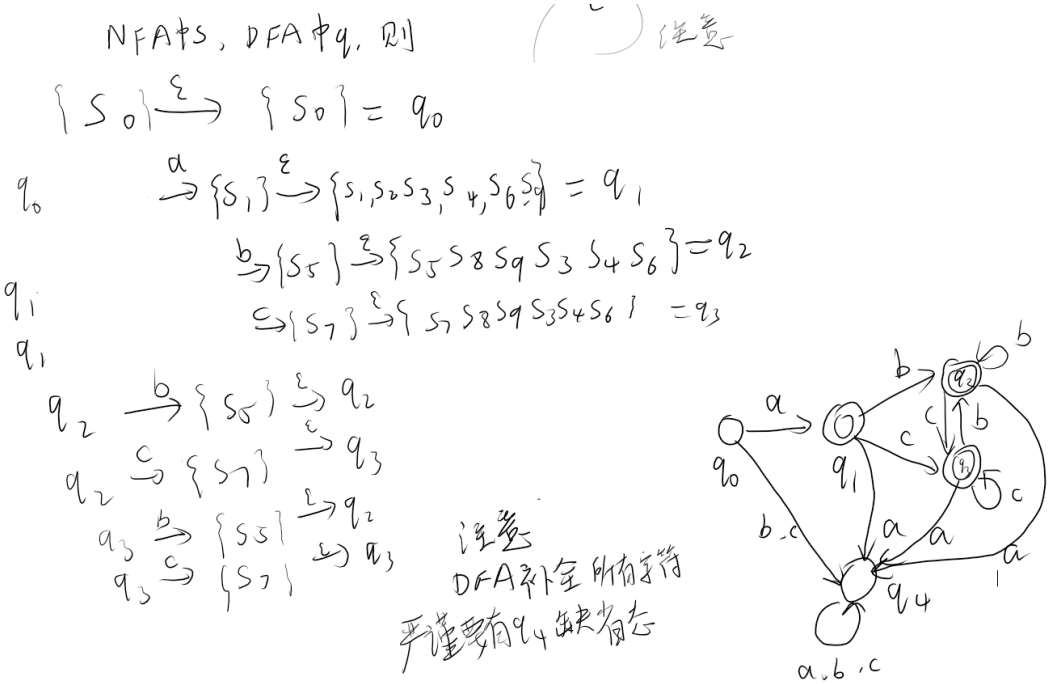

```
q0* <- eps_closure (s0)
S* <- q0*
workList <- q0*
while (workList != []) {
    remove q from workList
    foreach (character c) {
        s <- eps_closure (D(q, c))
        //D(q, c) = {D(si, c)}, si belongs to q
        D*(q, c) <- s
        if (s \not\in S*)
            add s to S* and workList
    }
}
```

1. $q0^* = \{s0\}$
2. $q0^* \xrightarrow{a} s1 \xrightarrow{\varepsilon} \{s1, s2, s3, s4, s6, s9\} = q1$
3. $q1 \xrightarrow{b} s5 \xrightarrow{\varepsilon} \{s5, s8, s9, s3, s4, s6\} = q2$

...



51



先构造子集，也就是状态的所有组合中，可能出现的状态集合，这通过模拟FA的运行得到，也就是：

先：最开始的q_0，或，之后的遇见字母的状态转移，得到初始状态集合

再：初始状态集合 + ε转移扩展 = 该状态集合

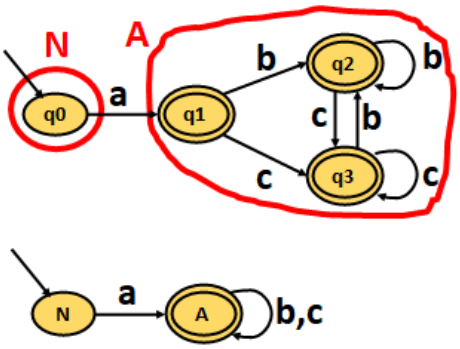
这个也在形式语言与自动机学过的。

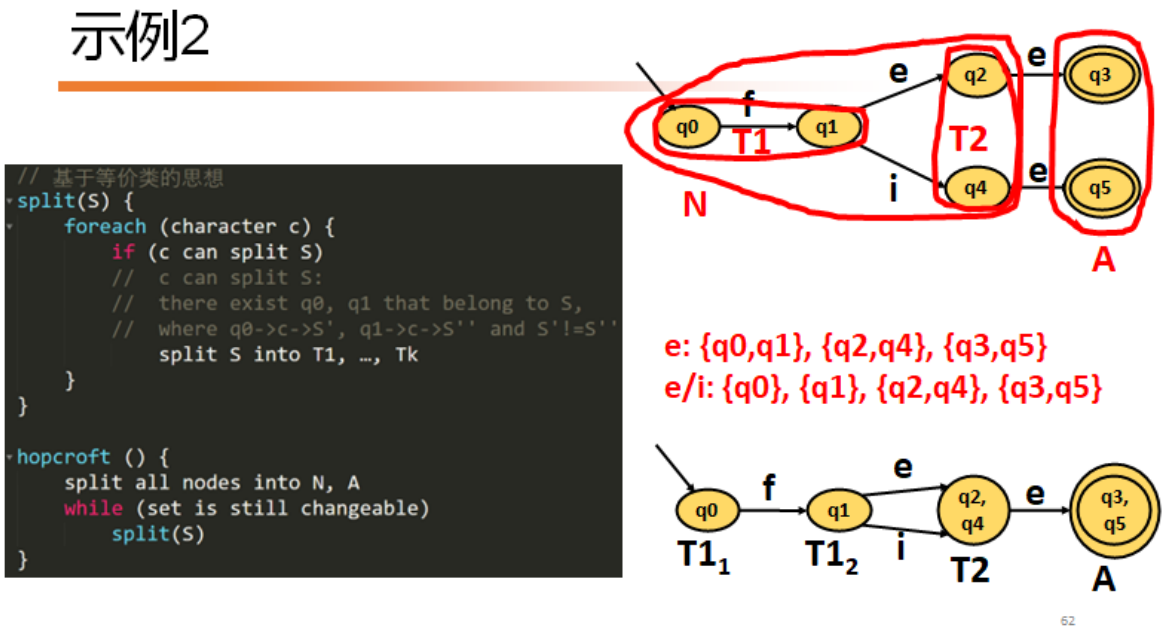
- 极小化DFA

示例1

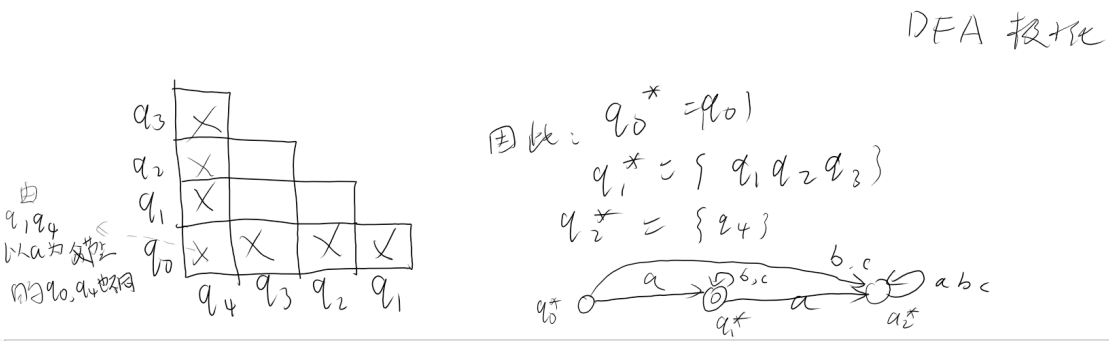
```
// 基于等价类的思想
split(S) {
    foreach (character c) {
        if (c can split S)
        //  c can split S:
        //  there exist q0, q1 that belong to S,
        //  where q0->c->S', q1->c->S'' and S'!=S''
            split S into T1, …, Tk
    }
}

hopcroft () {
    split all nodes into N, A
    while (set is still changeable)
        split(S)
}
```



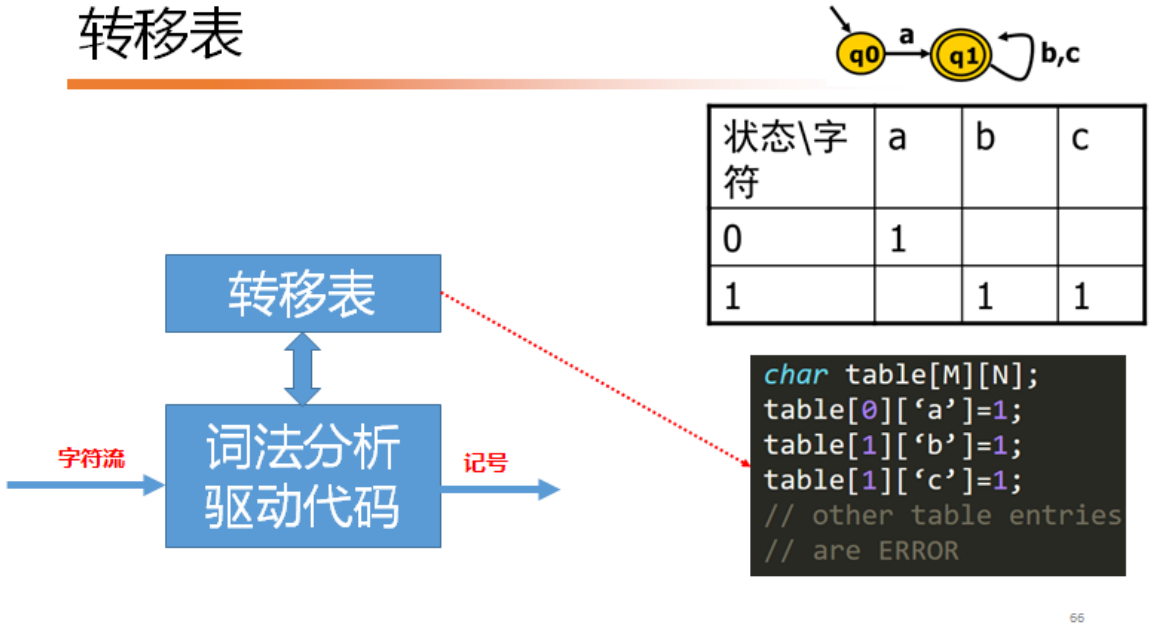


形式语言与自动机讲过，用列表法，先区分是否终态，再对每一个区分对标记父节点可区分，最后把不可区分的合并。



- DFA的代码实现

介绍了一种方法

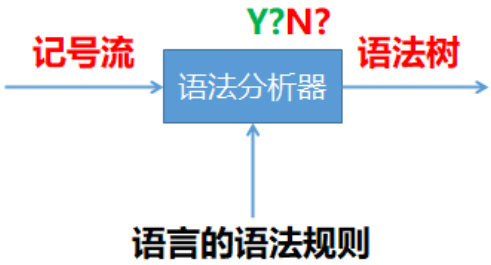


具体实现上，匹配最长的为好

第三章：自底向上重点，LALR不考

- 语法分析器

语法分析器的任务



6

例子：语法错误处理

```
if ((x > 5)
  y = "hello"
else
  z = 1,
```

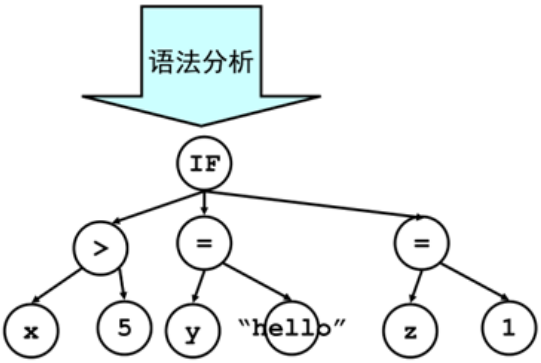
语法分析

```
Syntax Error: line 1, missing )
Syntax Error: line 2, missing ;
Syntax Error: line 4, expecting ; but got ,
```

7

例子：语法树构建

```
if ((x > 5)
   y = "hello"
else
   z = 1,
```



- CFG

用正则表达式?

- 正则表达式**不适用**于描述程序语法规则

- 例如：
- 算式：整数和运算符+、-构成
  - $[0\text{-}9]^+((+|-)[0\text{-}9]^+)^+$
- **可能带括号**的算式?
  - $((|\varepsilon))\,[0\text{-}9]^+((+|-)\,[0\text{-}9]^+)^+\,((\varepsilon|))$

9+9　　(9+9)

(9+9 ?　　9+9) ?

记不住前面出没出现过左括号！

示例

- 自然语言中的句子的典型结构：
  - 主语 谓语 宾语
  - 名词 动词 名词

语法规则

- 例子：
  - 名词：{羊、老虎、草、水}
  - 动词：{吃、喝}
- 句子：
  - 羊吃草，羊喝水，老虎吃羊，。。。
  - 羊吃老虎，老虎吃草，。。。

上下文无关文法：定义

- 上下文无关文法**G**是一个四元组：

$$G = (T, N, P, S)$$

  - 其中**T**是终结符集合
  - **N**是非终结符集合
  - **P**是一组产生式规则
    - 每条规则的形式：$X \rightarrow \beta_1\beta_2\ldots\beta_n$
      - 其中$X \in N$，$\beta_i \in (T \cup N)$
  - **S**是唯一的开始符号（非终结符）
    - $S \in N$

上下文无关文法的例子#1

```
S -> N V N
N -> s
   | t
   | g
   | w
V -> e
   | d
```

- **G = (T, N, P, S)**
- 非终结符：N={S, N, V}
- 终结符：T={s,t,g,w,e,d}
- 开始符号：S
- 产生式规则集合：
  - S -> N V N
  - N -> s
  - | t
  - | g
  - | w
  - V -> e
  - | d

上下文无关文法的例子#2

```
E -> num
   | id
   | E+E
   | E*E
```

- G = (T, N, P, S)
- 非终结符：N={E}
- 终结符：T={num, id, +, *}
- 开始符号：E
- 产生式规则集合：
  - E -> num
  - | id
  - | E+E
  - | E*E

19

上下文无关文法的核心特征就是左边只有一个非终结符，这样子就和周围无关。注意右边有可能是空，所以可能更短。

- 推导

推导

- 给定文法G，从G的开始符号S开始，用产生式的右部替换左侧的**非终结符**
- 此过程不断重复，直到**不出现非终结符为止**
- 最终的串称为**句子**

- **示例：**

```
S -> N V N
N -> s
   | t
   | g
   | w
V -> e
   | d
```

```
S -> N V N
  -> s V N
  -> s e N
  -> s e g
```

21

最左推导和最右推导

- 最左推导：每次总是选择**最左侧**的符号进行替换
- 最右推导：每次总是选择**最右侧**的符号进行替换

- **示例：**

```
S -> N V N
N -> s
   | t
   | g
   | w
V -> e
   | d
```

最左推导：

```
S -> N V N
  -> s V N
  -> s e N
  -> s e g
```

最右推导：

```
S -> N V N
  -> N V g
  -> N e g
  -> s e g
```


语法分析：形式化定义

- 给定文法**G**和句子**S**，语法分析要回答的问题：

  是否存在对句子s的推导?

- **示例：**

```
S -> N V N
N -> s
   | t
   | g
   | w
V -> e
   | d
```

S = s e s 

S = s e e 

最左/右推导，同一个句子还有两个实现方式，就叫二义性

- 分析树

分析树

- 推导可以表达成树状结构：**分析树**
  - 不论推导所用的顺序（最左、最右、其他），都可以表示
- 特点
  - 树中的每个**内部节点**代表**非终结符**
  - 每个**叶子节点**代表**终结符**
  - 每一步推导代表如何从**父节点**生成它的直接**孩子节点**

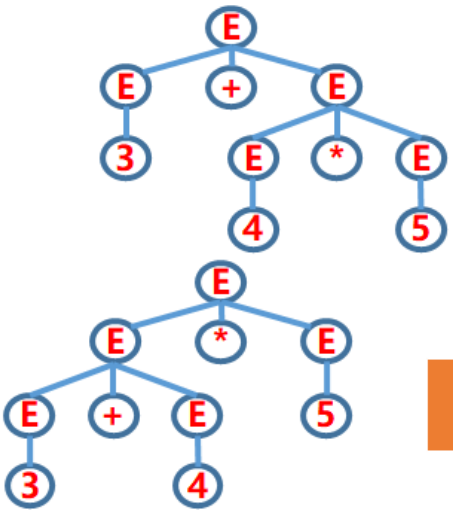
分析树：示例

- 推导

```
E -> E+E
  -> 3+E
  -> 3+E*E
  -> 3+4*E
  -> 3+4*5
```

```
E -> E*E
  -> E+E*E
  -> 3+E*E
  -> 3+4*E
  -> 3+4*5
```

- 分析树




```
E -> num
   | id
   | E+E
   | E*E
```

3+4*5

推导这个句子

分析树的含义取决于树的**后序遍历**

- 自顶向下分析

自顶向下分析的算法思想

- 语法分析：给定文法G和句子s，回答s是否能够从G推导出来？
- 基本算法思想：从G的开始符号出发，随意推导出某个句子t，比较t和s
  - 若t==s，则回答“是”
  - 若t!=s，则?
- 因为这是从开始符号出发推出句子，因此称为**自顶向下分析**
  - 对应于**分析树**自顶向下的构造顺序

示例

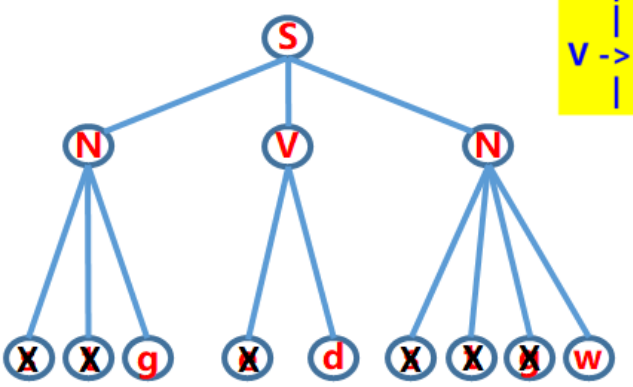
g d w 推导这个句子

S -> N V N
N -> s
| t
| g
| w
V -> e
| d

S -> N V N
-> s V N
-> s e N
-> s e g

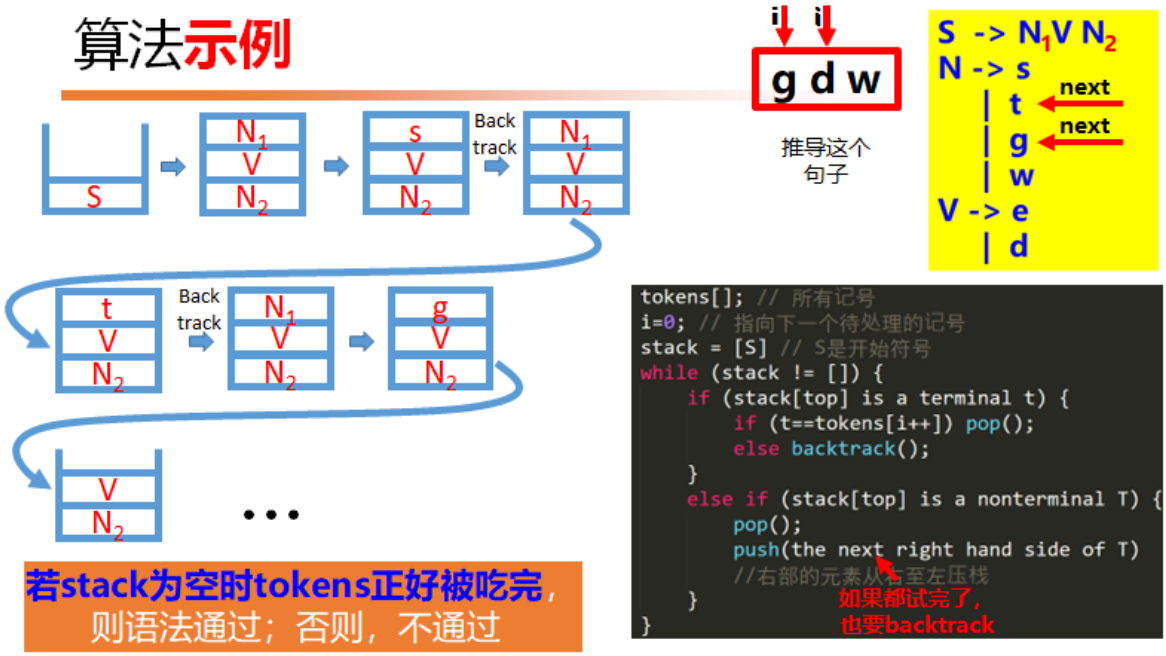
X

S -> N V N
-> g V N
-> g d N
-> g d w



虽然感觉效率很低，但是当长度确定，可能性是有限的，可以搜索。

上面的图说的是，碰巧到gdw时匹配到。

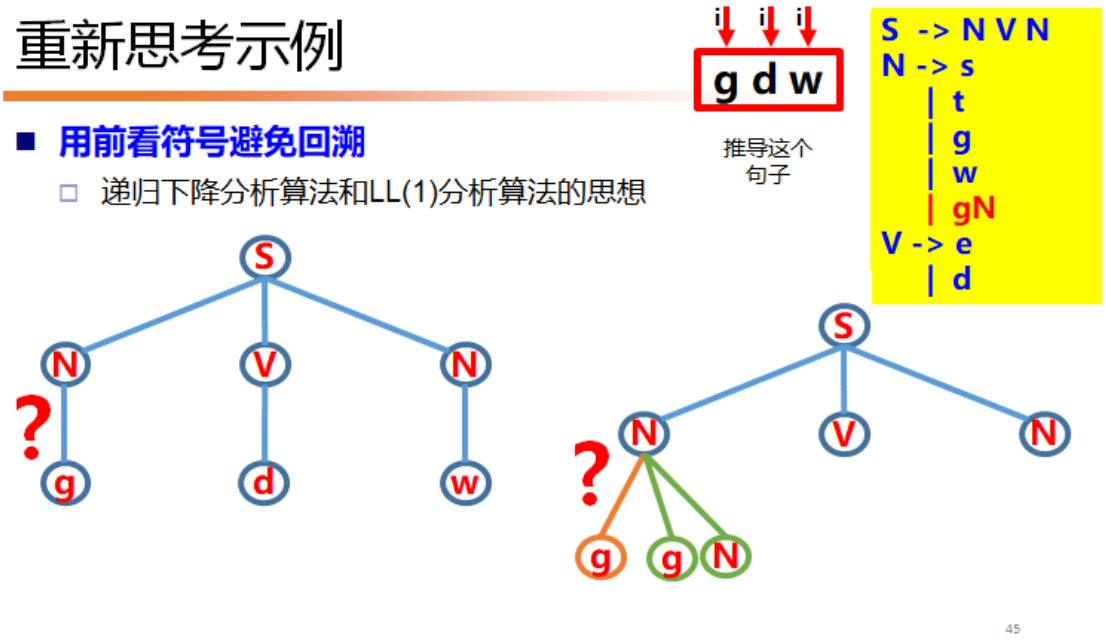


用栈的方式推导，把各子树无关地分别看待，匹配到一个就结束一个，则确定了父派生方式，这是错误的。

显然，如果是单线操作，是会出错的，比如可能a开头的有两个，就可以走到了错误的路线。

要么改进，要么非确定性。之后有改进。

- 改进速度：递归下降分析，加前看



递归下降分析算法

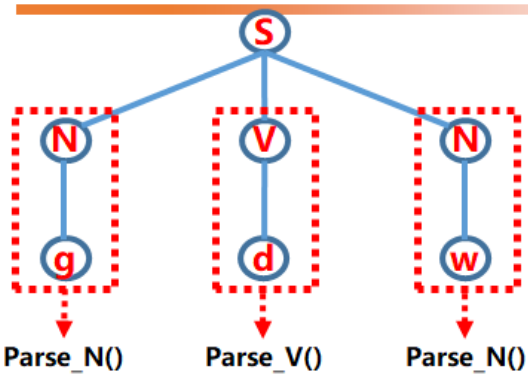
- 也称为**预测分析**
  - 分析高效（线性时间）
  - 容易实现（方便手工编码）
  - 错误定位和诊断信息准确
  - 被很多开源和商业的编译器所采用
    - GCC 4.0，LLVM，。。。
- 算法基本思想
  - 将每一个**非终结符**转变成一个**子函数**
  - 用**前看符号**指导产生式规则的选择

示例

g d w

推导这个句子

```
S -> N V N
N -> s
   | t
   | g
   | w
V -> e
   | d
```



- 算法基本思想
  - 将每一个**非终结符**转变成一个**子函数**
  - 用**前看符号**指导产生式规则的选择

算法示例

推导这个句子 **g d w**

```
S -> N V N
N -> s
   | t
   | g
   | w
V -> e
   | d
```

```
parse_S() {
    parse_N();
    parse_V();
    parse_N();
}

parse_N() {
    token = tokens[i++];
    if (token==s||token==t||token==g||token==w)
        return;
    error("…");
}
```

```
parse_V() {
    token = tokens[i++];
    if (token==e||token==d)
        return;
    error("…");
}
```

示例

```
parse_X() {
    token = nextToken();
    switch(token)
    if (token==a) {
        move token;
        parse_B();
        if(token==c) {
            move token;
            parse_D();
        }
        else
            error ("...");
    }
    else if (token==b) {
        move token;
        parse_D();
    }
    else
        error ("...");
}
```

X -> aBcD
| bD
B -> ...
D -> ...

```
parse_B() {
    token = nextToken();
    if :
    ...
    else: error ("...");
}

parse_D() {
    token = nextToken();
    if :
    ...
    else: error ("...");
}
```

50

- 改进：增加适用范围。这样子可以纠正之前的出错。

预处理#1：提取左公因子

- 如果：

$$A \to aB_1 \mid aB_2 \mid aB_3 \mid \dots$$

- 那么：
  - a就为左公因子

前看符号无法指导分支的选择！

- 解决办法：**提取左公因子**
  - 将产生式变换为

$$A \to aA'$$
$$A' \to B_1 \mid B_2 \mid B_3 \mid \dots$$

其中，$B_1, B_2, B_3 \dots$，没有左公因子

```
parse_A() {
    token = nextToken();
    if (token==a)
        parse_Aprime();
    else
        error ("...");
}
```

```
parse_Aprime() {
    if (token==B1)
        parse_B1();
    else if (token==B2)
        parse_B2();
    else if (token==B3)
        parse_B3();
    ...
    else: error ("...");
}
```

51

预处理#2：消除左递归

- 如果：A -> Aa
- 那么：该产生式是左递归的

无法递归调用产生式右部中非终结符的分析函数

- 解决办法：消除左递归
  - 将产生式变换为

A -> Aa|B ⇒

A -> BA'
A' -> aA'|ε

- 示例

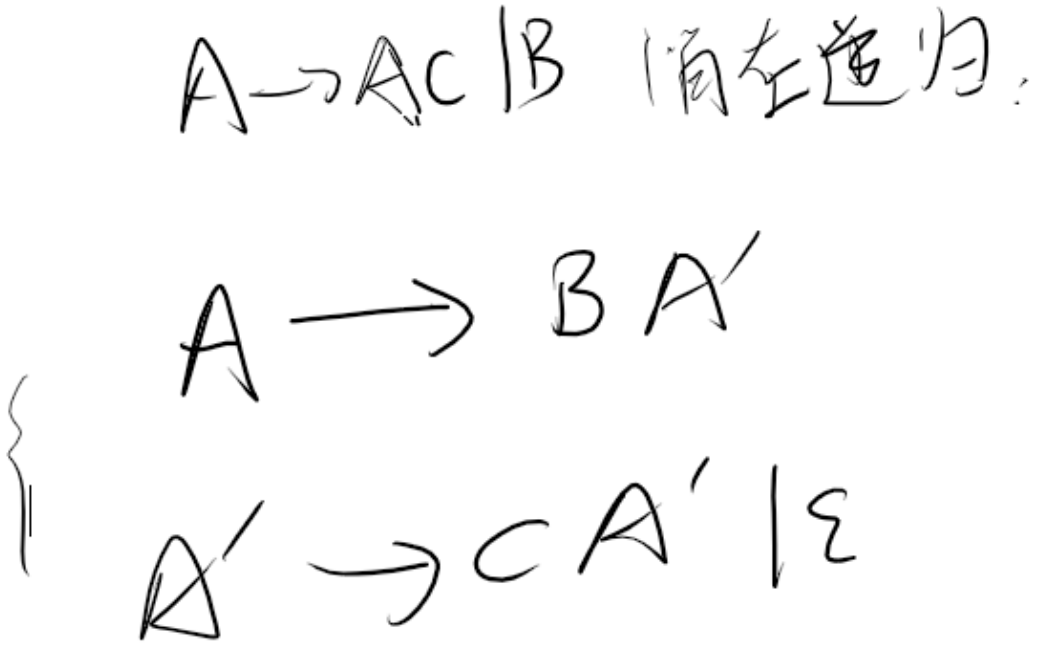
E -> E+T|T'

⇓

T' + T + T ....

⇓

E -> T' E'
E' -> +TE'|ε



预处理#2：消除左递归

- 示例#2

```
E -> E+T
   | T
T -> T*F
   | F
F -> num
   | id
```

⇒

```
E  -> TE'
E' -> +TE' |ε
T  -> FT'
T' -> *FT' |ε
F  -> num
   | id
```

注意符合逻辑地，转换左递归。

- LL1

一般条件下的FIRST集算法

FIRST_S(β_1... β_n) =
FIRST(N), if β1 == N;
{a}, if β1 == a.

近似版本

- FIRST(N)
  - 从非终结符N开始推导得出的句子开头的所有可能终结符集合
  - 计算公式:
    - 对 N -> a ... FIRST(N) ∪= {a}
    - 对 N -> M ... FIRST(N) ∪= FIRST(M)

```
0: Z -> d {d}
1:      | X Y Z Z{c,a}
2: Y -> c {c}
3:      | ε {}
4: X -> Y {c}
5:      | a {a}
```

- 计算一个产生式的FIRST集：FIRST_S(β_1... β_n)
  - 一般情况下需要知道某个非终结符**是否可以推出空串**
    - **NULLABLE集**
  - 并且一般需要知道在某个非终结符**后面跟着什么符号**
    - **跟随集FOLLOW**

通用版本

70

NULLABLE集合

- 归纳定义：
- 非终结符X属于集合NULLABLE，当且仅当：
  - 基本情况：
    - **X -> ε|...**
  - 归纳情况：
    - **X -> Y_1 ... Y_n**
      - **Y_1, ..., Y_n 是n个非终结符，且都属于NULLABLE集**

71

NULLABLE集合算法：示例

```
NULLABLE = {};

while (NULLABLE is still changing) {
    foreach (production p: X-> β) {
        if (β == epsilon)
            NULLABLE ∪= {X}
        if (β== Y1 … Yn) {
            if (Y1 belongs_to NULLABLE && … && Yn belongs_to NULLABLE)
                NULLABLE ∪= {X}
        }
    }
}
```

```
0: Z -> d
1:    | X Y Z Z
2: Y -> c
3:    | ε
4: X -> Y
5:    | a
```

Round	0	1	2	3
NULLABLE	{}	{Y, X}	{Y, X}	

伪代码：

```
NULLABLE = {};
while (NULLABLE is still changing) {
    foreach (production p: X-> β) {
        if (β == epsilon)NULLABLE U=  {x}
        if (β== Y1 ... Yn) {
            if (Y1 belongs_to NULLABLE && ... && Yn belongs_to NULLABLE) NULLABLE
U= {X}
        }
    }
}
```

FIRST(N)集的不动点算法：示例

```
0: Z -> d
1:    | X Y Z Z
2: Y -> c
3:    | ε
4: X -> Y
5:    | a
```

- NULLABLE集

NULLABLE	{Y, X}

- FIRST(N)集

N\Round	0	1	2
Z	{}	{d}	{d,c,a}
Y	{}	{c}	{c}
X	{}	{c,a}	{c,a}

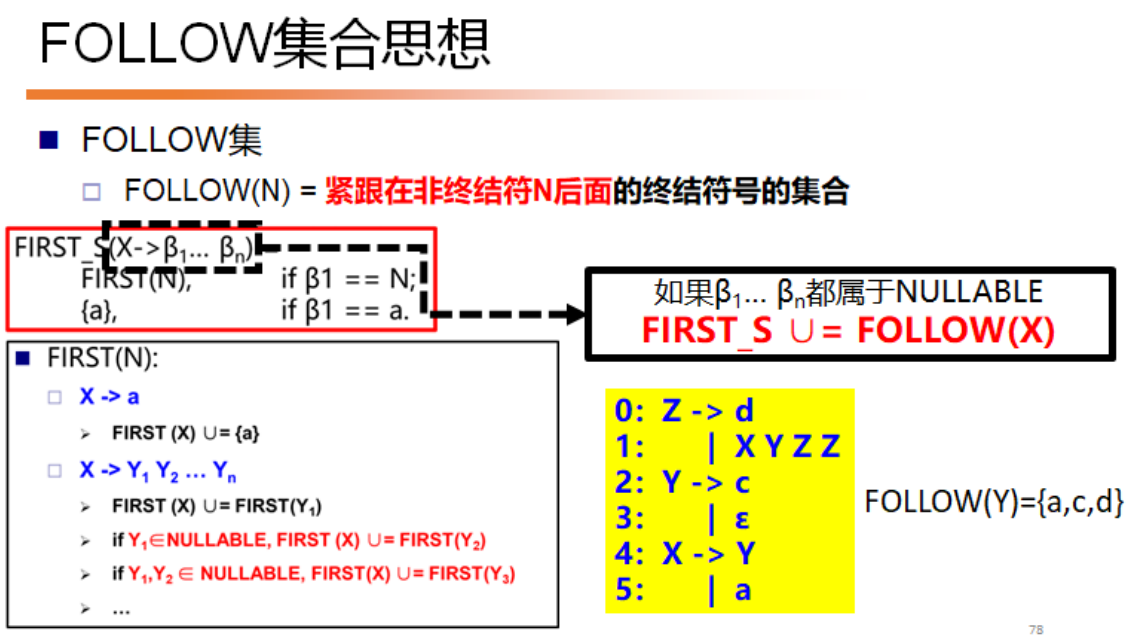
```
foreach (nonterminal N)
    FIRST(N) = {}

while(some set is changing) {
    foreach (production p: N-> β1 … βn) {
        foreach (βi from β1 upto βn) {
            if (βi == a …) {
                FIRST(N) ∪= {a}
                break
            }
            if ( βi== M …) {
                FIRST(N) ∪= FIRST(M)
                if (M is not in NULLABLE) break;
            }
        }
    }
}
```

伪代码：

```
foreach (nonterminal N)
    FIRST(N)={}
while(some set is changing) {
    foreach (production p: N-> β1 ... βn) {
        foreach (βi from β1 upto βn) {
```

```
            if (βi == a ...) {
                FIRST(N) u={a}
                break
            }
            if ( βi== M ...) {
                FIRST(N) u= FIRST(M)
                if (M is not in NULLABLE)break
            }
        }
    }
}
```



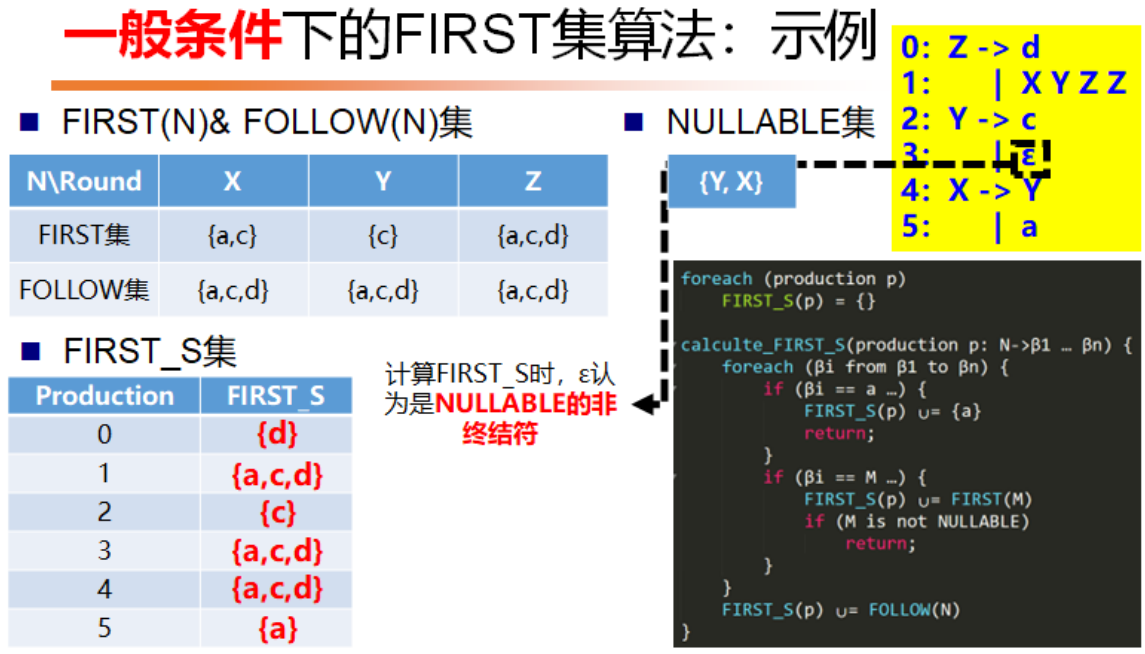


伪代码：

```
foreach (nonterminal N)
    FOLLOW(N) = {}

while(some set is changing) {
```

```
    foreach (production p: N->β1..βn) {
        temp = FOLLOW(N)
        foreach （βi from βn downto β1） { // 逆序!
            if (βi == a ..) temp = {a};
            if (βi == M....){
                FOLLOW(M) u= temp
                if (M is not NULLABLE) temp = FIRST(M)
                else temp u= FIRST(M)
            }
        }
    }
}
```



伪代码：

```
foreach (production p)
    FIRST_S(p) = {}
calculte_FIRST_S(production p: N->β1 ... βn){
    foreach (βi from β1 to βn) {
        if (βi == a ...) {
            FIRST_S(p) u= {a}
            return;
        }
        if (βi == M ...) {
            FIRST_S(p) U= FIRST(M)
            if (M is not NULLABLE)return;
        }
    }
    FIRST_S(p) u= FOLLOW(N)
}
```

LL(1)文法中的冲突：示例

- 示例一：

LL(1)分析表

N\T	s	t	g	w	e	d
S	0	0	0	0		
N	1	2	3	4,5		
V					6	7

```
0: S -> N V N
1: N -> s
2:    | t
3:    | g
4:    | w
5:    | wN
6: V -> e
7:    | d
```

- 示例二：

LL(1)分析表

N\T	a	c	d
Z	1	1	0,1
Y	3	2,3	3
X	4,5	4	4

```
0: Z -> d
1:    | X Y Z Z
2: Y -> c
3:    | ε
4: X -> Y
5:    | a
```


消除文法中的冲突使其满足LL(1)分析：#1

- 若FIRST(α)与FIRST(β)**相交**
  - **提取α与β的左公因子**

$A \to \alpha|\beta$ ⇒ $A \to \gamma A'$, $A' \to \alpha_1|\beta_2$

其中，α1,β2**没有**左公因子

- 示例：

```
0: S -> N V N
1: N -> s
2:    | t
3:    | g
4:    | w
5:    | wN
6: V -> e
7:    | d
```

⇒

```
0: S -> N V N
1: N -> s
2:    | t
3:    | g
4:    | wN'
5: N'-> ε
6:    | N
7: V -> e
8:    | d
```

LL(1)分析表

N\T	s	t	g	w	e	d
S	0	0	0	0		
N	1	2	3	4,5		
V					7	8

N\T	s	t	g	w	e	d
S	0	0	0	0		
N	1	2	3	4		
N'	6	6	6	6	5	5
V					7	8

消除文法中的冲突使其满足LL(1)分析：#2

- 若α与β**都能推导出ε**
  - **删掉α或β某一个产生式中的ε**

$A \to \alpha|\beta$ ⇒ $A \to (\alpha-\varepsilon) \mid \beta$

α-ε表示从α中删除ε后的产生式

- 示例：

```
0: Z -> aZ
1:    | Y
2:    | ε
3: Y -> c
4:    | ε
```

⇒

```
0: Z -> aZ
1:    | c
2:    | ε
3: Y -> c
4:    | ε
```

LL(1)分析表

N\T	a	c	ε
Z	0	1	1,2
Y		3	4

N\T	a	c	ε
Z	0	1	2
Y		3	4

消除文法中的冲突使其满足LL(1)分析：#3

- 若β能**推导出ε**，且FIRST(α)与FOLLOW(A)**相交**；或若α能**推导出ε**，且FIRST(β)与FOLLOW(A)**相交**
  - **展开产生式，消除左递归或提取左公因子**
    - Case1: 提取左公因子

- 示例：

```
0: Z -> d
1:    | Xad
2: X -> ab
3:    | ε
```

```
0: Z -> d
1:    | (ab|ε)ad
```

```
0: Z -> d
1:    | abad
2:    | ad
```

```
0: Z -> d
1:    | aZ'
2: Z'-> bad
3:    | d
```

LL(1)分析表

N\T	a	b	d
Z	1		0
X	2,3		

N\T	a	b	d
Z	1		0
Z'		2	3

消除文法中的冲突使其满足LL(1)分析：#3

- 若β能**推导出ε**，且FIRST(α)与FOLLOW(A)**相交**；或若α能**推导出ε**，且FIRST(β)与FOLLOW(A)**相交**
  - **展开产生式，消除左递归或提取左公因子**
    - Case2: 消除左递归

- 示例：

```
0: Z -> d
1:    | Ya
2: Y -> Z
3:    | ε
```

```
0: Z -> d
1:    | (Z|ε)a
```

```
0: Z -> d
1:    | a
2:    | Za
```

```
0: Z -> dZ'
1:    | aZ'
2: Z'-> aZ'
3:    | ε
```

LL(1)分析表

N\T	a	d
Z	1	0,1
Y	2,3	2

N\T	a	d
Z	1	0
Z'	2	

消除文法中的冲突使其满足LL(1)分析：#3

- 若β能**推导出ε**，且FIRST(α)与FOLLOW(A)**相交**；或若α能**推导出ε**，且FIRST(β)与FOLLOW(A)**相交**
  - **展开产生式，消除左递归或提取左公因子**
    - Case3: 消除左递归（失败）

- 示例：

```
0: Z -> d
1:    | YZa
2: Y -> c
3:    | ε
```

```
0: Z -> d
1:    | (c|ε)Za
```

```
0: Z -> d
1:    | cZa
2:    | Za
```

```
0: Z -> dZ'
1:    | cZaZ'
2: Z'-> aZ'
3:    | ε
```

LL(1)分析表

N\T	a	c	d
Z		1	0,1
Y		2,3	3

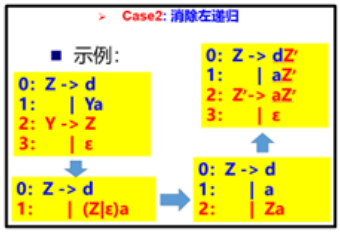
N\T	a	c	d
Z		1	0
Z'	2,3		

凉凉！
FOLLOW(Z')也包含a

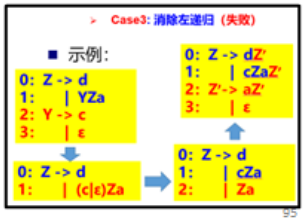
无回溯文法的不可判定性

- 提取左公因子或消除左递归通常可以将CFG变换为无回溯文法





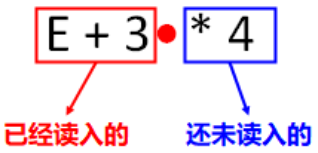
- 但**并非所有的CFG都有对应的无回溯文法**
  - 对于任意的CFG，其是否存在无回溯文法是**不可判定的**



- LR

点记号

- 为了方便标记语法分析器**已经读入了多少输入**，我们可以引入一个点记号 ●



自底向上分析的基本思想

```
0: S -> E
1: E -> E+T
2:    | T
3: T -> T*F
4:    | F
5: F -> num
```

规约：比对产生式，看是否能收缩成左部

2 + 3 * 4		2 • + 3 * 4
F + 3 * 4	P5	F • + 3 * 4
T + 3 * 4	P4	T • + 3 * 4
E + 3 * 4	P2	E + 3 • * 4
E + F * 4	P5	E + F • * 4
E + T * 4	P4	E + T * 4 •
E + T * F	P5	E + T * F •
E + T	P3	E + T •
E	P1	E •
S	P0	S •

另外的写法

```
0: S -> E
1: E -> E+T
2:    | T
3: T -> T*F
4:    | F
5: F -> num
```

规约：比对产生式，看是否能收缩成左部

左边是一个栈。每次读入记号时入栈（即移进）；在栈顶若干元素匹配某产生式时，弹出产生式右部、压入产生式左部（即归约）

2	2 + 3 * 4
F	• + 3 * 4
T	• + 3 * 4
E	• + 3 * 4
E +	• + 3 * 4
E + 3	• 3 * 4
E + F	• * 4
E + T	• * 4
E + T *	• * 4
E + T * 4	• 4
E + T * F	•
E + T	•
E	•
S	•

显然，上面的也还是不正确的，要改进。例如不应该遇到就直接归约。

自底向上推导（LR）的步骤

- 需要两个步骤：
  - **移进（shift）**一个记号到栈顶上，或者
  - **归约（reduce）**栈顶上的n个符号（某产生式的右部） 到左部的非终结符
    - 对产生式A -> $\beta_1 \dots \beta_n$
      - 如果$\beta_n \dots \beta_1$在栈顶上，则弹出$\beta_n \dots \beta_1$
      - 压入A

核心问题：
如何确定移进和归约的时机？

LR分析表

- 在当前**状态**和当前**输入记号**时，应该执行什么**操作**

增广文法

0: S'-> S$
1: S -> x x T
2: T -> y

状态栈　记号栈

	动作（ACTION）			转移（GOTO）	
状态\符号	x	y	$	S	T
1	s2			g6	
2	s3				
3		s4			g5
4	r2	r2	r2		
5	r1	r1	r1		
6			accept		

- *sn*：**移进**
  - 吃进当前记号，压入记号栈
  - 转移到状态n，把状态n压入状态栈
  - 挪到下一个输入记号

LR分析表

- 在当前**状态**和当前**输入记号**时，应该执行什么**操作**

0: S'-> S$
1: S -> x x T
2: T -> y

状态栈　记号栈

	动作（ACTION）			转移（GOTO）	
状态\符号	x	y	$	S	T
1	s2			g6	
2	s3				
3		s4			g5
4	r2	r2	r2		
5	r1	r1	r1		
6			accept		

- *rn*：**归约**
  - 从记号栈中弹出产生式n的所有右部记号，同时从状态栈中弹出相等数量的状态
  - 将产生式n的左部压入记号栈
  - 根据当前状态栈中的栈顶状态，查询LR分析表的转移部分

LR分析表

- 在当前**状态**和当前**输入记号**时，应该执行什么**操作**

0: S'-> S$
1: S -> x x T
2: T -> y

状态栈　记号栈

	动作（ACTION）			转移（GOTO）	
状态\符号	x	y	$	S	T
1	s2			g6	
2	s3				
3		s4			g5
4	r2	r2	r2		
5	r1	r1	r1		
6			accept		

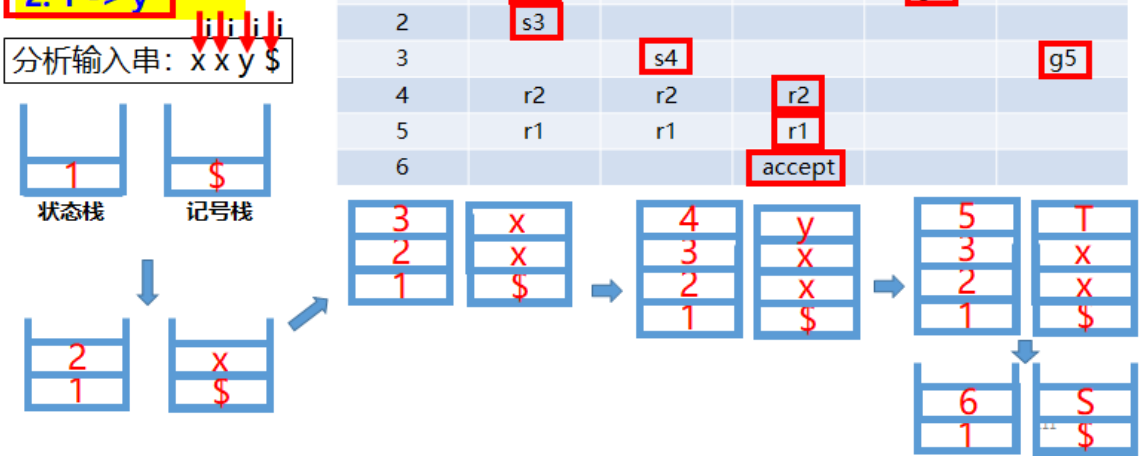
- g*n*：**转移**
  - 转移到状态n，把状态n压入状态栈
- accept：**接受**
  - 停止分析并报告成功
- 空白：**出错**
  - 停止分析并报告错误

LR分析算法：**示例**

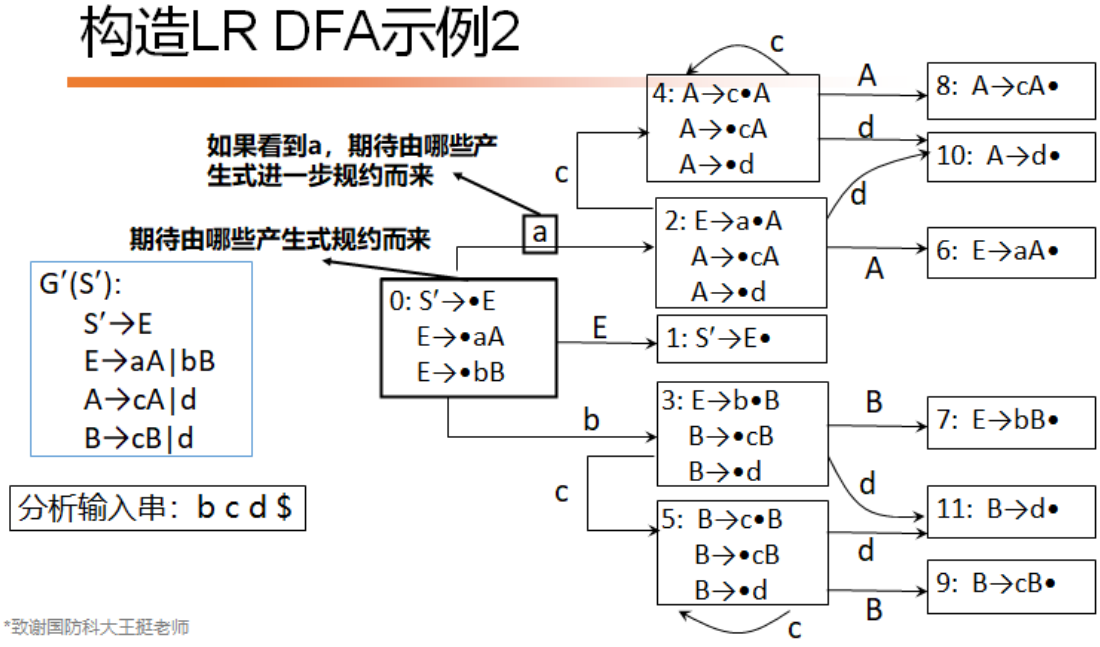
0: S' -> S$
1: S -> x x T
2: T -> y

分析输入串：x x y $

	动作（ACTION）			转移（GOTO）	
状态\符号	x	y	$	S	T
1	s2			g6	
2	s3				
3		s4			g5
4	r2	r2	r2		
5	r1	r1	r1		
6			accept		



构造LR DFA示例2



*致谢国防科大王挺老师

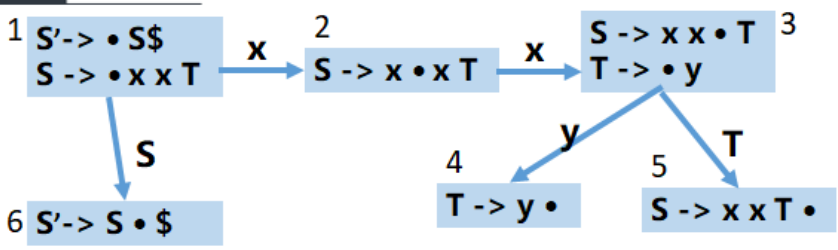
LR(0)分析表构造算法—DFA构造算法

```
State0=closure(S'→•S$) // the initial
STATE_SET = {State0} // all states
while(STATE_SET is changing) {
    foreach (unvisited state I in STATE_SET) {
        foreach (grammar symbol X) {
            D=goto(I, X)
            if(D is not empty and not in STATE_SET) {
                STATE_SET ∪= D }
                Add an edge X from state I to state D
        }
        set state I as visited
    }
}
```

```
goto (C, x)
  temp = {}          // a set
  foreach (C's item i: A -> β• x γ)
    temp ∪= {A -> βx • γ}
  return closure (temp)
```

```
closure (C)
  while(C is still changing)
    foreach (C's item i: A -> β• B γ)
      C ∪= {B -> …}
```

■ 示例：



看伪代码就明白意思了。

closure的意思是，·后面是非终结符就加进来，因为可能要用到这个来归约。

goto的意思是，在状态c对于输入x，会到什么状态，里面要先查派生规则，再用到closure。

主程序里面，就从初始状态开始，构造了DFA的状态和转移。主程序中有一个已经被改正的错误，也就是，之前形成了的状态也是要加边。

构造DFA示例

1. 拓（增）广文法 (augmented grammar)

当且仅当分析器使用$E' \to E$归约时，宣告分析成功

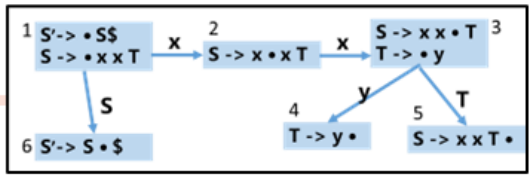
$$E' \to E$$
$$E \to E + T \mid T$$
$$T \to T * F \mid F$$
$$F \to (E) \mid \text{id}$$

*致谢中科大李诚老师

123

注意增广文法，添加E'-->E，作为DFA的根节点，用了这个归约就成功分析时结束。当然，相应地添加一个输入的终结符$也是更方便的，不添加也没什么不行。参考老师的还是添加$，用到归约改成读入$即可。

LR(0)分析表构造算法—根据DFA生成LR(0)分析表



- 对每个DFA状态
  - 对转移到其他状态的**终结符边**，记为该状态上的**移进操作s*n***
  - 对转移到其他状态的**非终结符边**，记为该状态上的**转移操作g*n***
  - 若某状态中包含项目**A → α •**，记为该状态上对产生式**A → α**的**归约操作r*n***
    - 若为开始产生式，则对结束符动作记为accept

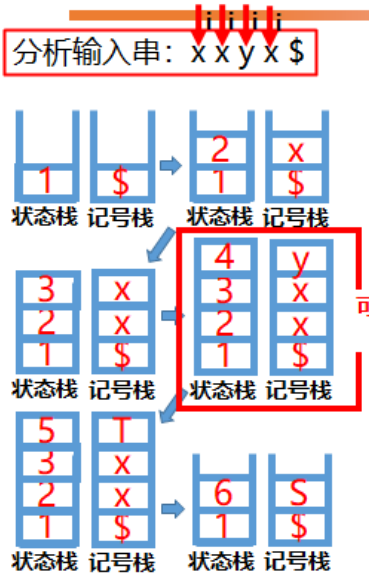
	动作（ACTION）			转移（GOTO）	
状态\符号	x	y	$	S	T
1	s2			g6	
2	s3				
3		s4			g5
4	r2	r2	r2		
5	r1	r1	r1		
6			accept		

135

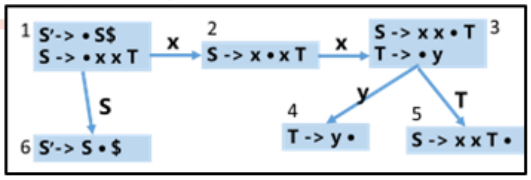
- SLR分析

LR(0)问题1：延迟错误发现时机

分析输入串：x x y x $



0: S'-> S$
1: S -> x x T
2: T -> y

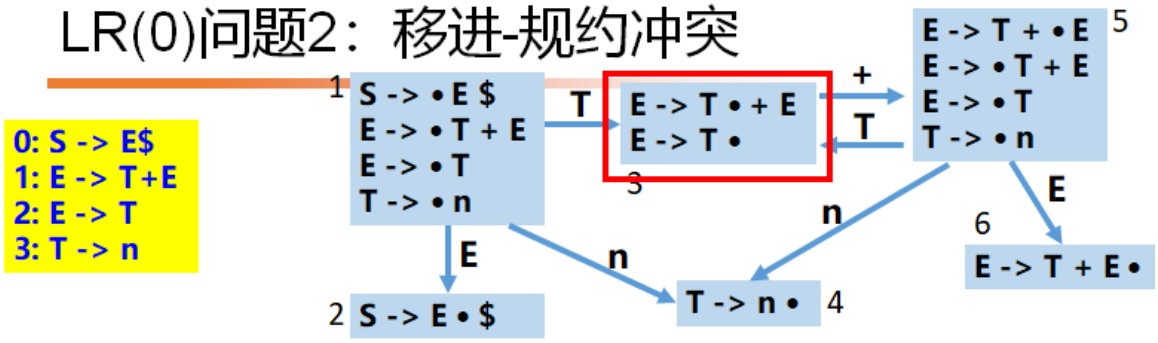


	动作（ACTION）			转移（GOTO）	
状态\符号	x	y	$	S	T
1	s2			g6	
2	s3				
3		s4			g5
4	r2	r2	r2		
5	r1	r1	r1		
6	X		accept		

可以在这一步就报错

144

LR(0)问题2：移进-规约冲突



	动作（ACTION）			转移（GOTO）	
状态\符号	n	+	$	E	T
1	s4			g2	g3
2			Accept		
3	r2	s5/r2	r2		
4	r3	r3	r3		
5		s4		g6	g3
6	r1	r1	r1		

LR有以上的问题，SLR可以通过措施减少出错的范围。

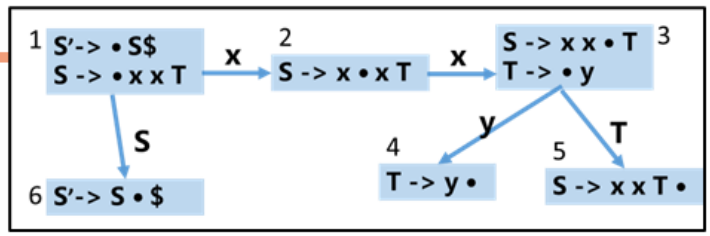
SLR分析算法

- 与LR(0)算法基本一样，仅区别于对**归约**表项的生成
  - DFA构造
  - 根据DFA生成~~LR(0)~~**SLR**分析表
    - 对转移到其他状态的**终结符边**，记为该状态上的**移进操作s*n***
    - 对转移到其他状态的**非终结符边**，记为该状态上的**转移操作g*n***
    - 若某状态中包含项目**A → α •**，~~记为该状态上对产生式A → α的归约操作r*n*~~
      - 若为开始产生式，则对结束符动作记为accept

仅对**属于FOLLOW(A)的输入记号**添加对产生式**A → α**的**归约操作r*n***

SLR：示例1

0: S'-> S$
1: S -> x x T
2: T -> y

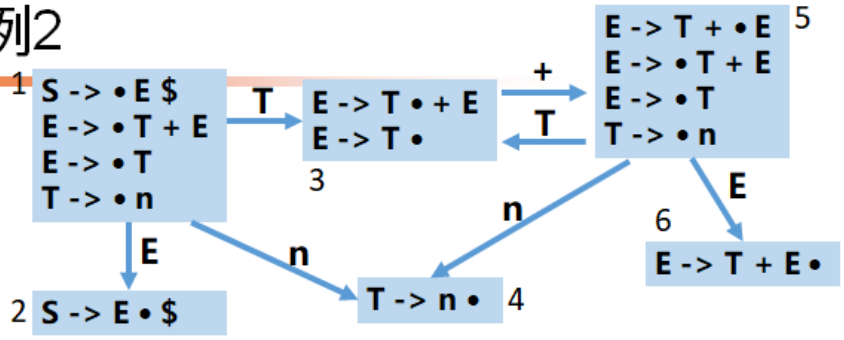


N	FOLLOW集
T	{$}
S	{$}

	动作（ACTION）			转移（GOTO）	
状态\符号	x	y	$	S	T
1	s2			g6	
2	s3				
3		s4			g5
4	~~r2~~	~~r2~~	r2		
5	~~r1~~	~~r1~~	r1		
6			accept		

SLR：示例2

0: S -> E$
1: E -> T+E
2: E -> T
3: T -> n



N	FOLLOW集
E	{$}
T	{+,$}

	动作（ACTION）			转移（GOTO）	
状态\符号	n	+	$	E	T
1	s4			g2	g3
2			Accept		
3		**s5**	**r2**		
4		**r3**	**r3**		
5	s4			g6	g3
6			**r1**		

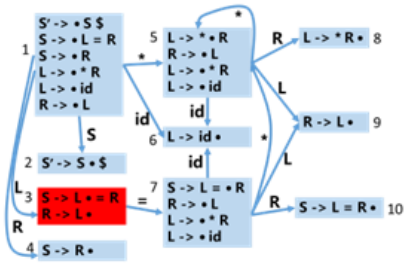
150

- LR1

SLR分析表中的冲突：示例

N	FOLLOW集
S	{$}
L	{=,$}
R	{=,$}

0: S' -> S$
1: S -> L = R
2: S -> R
3: L -> * R
4: L -> id
5: R -> L



	动作（ACTION）				转移（GOTO）		
状态\符号	id	=	*	$	S	L	R
1	s6		s5		g2	g3	g4
2				accept			
3		s7/r5		r5			
4				r2			
5	s6		s5			g9	g8
6		r4		r4			
7	s6		s5			g9	g10
8		r3		r3			
9		r5		r5			
10				r1			

154

SLR没有保证不冲突的措施，显然。

LR1可以更加减少这个范围。

LR(1)分析算法

LR(0)算法中的项目
[X → α • β]

LR(1)算法中的项目
[X → α • β, a]

a是前看符号

使用这条产生式归约时，下一个看到的输入记号应该是a

SLR：FOLLOW(X)

遇到产生式
后面可能跟的所有输入
都归约

LR(1)

在遇到
推导过程中
该产生式后面可能跟的输入
才归约

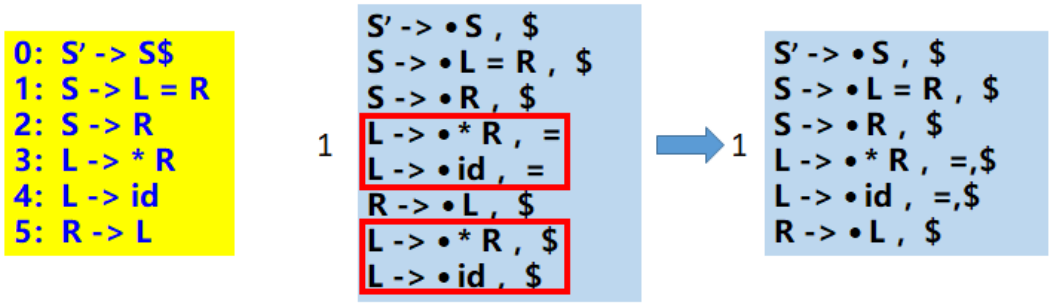
157

LR(1)分析算法

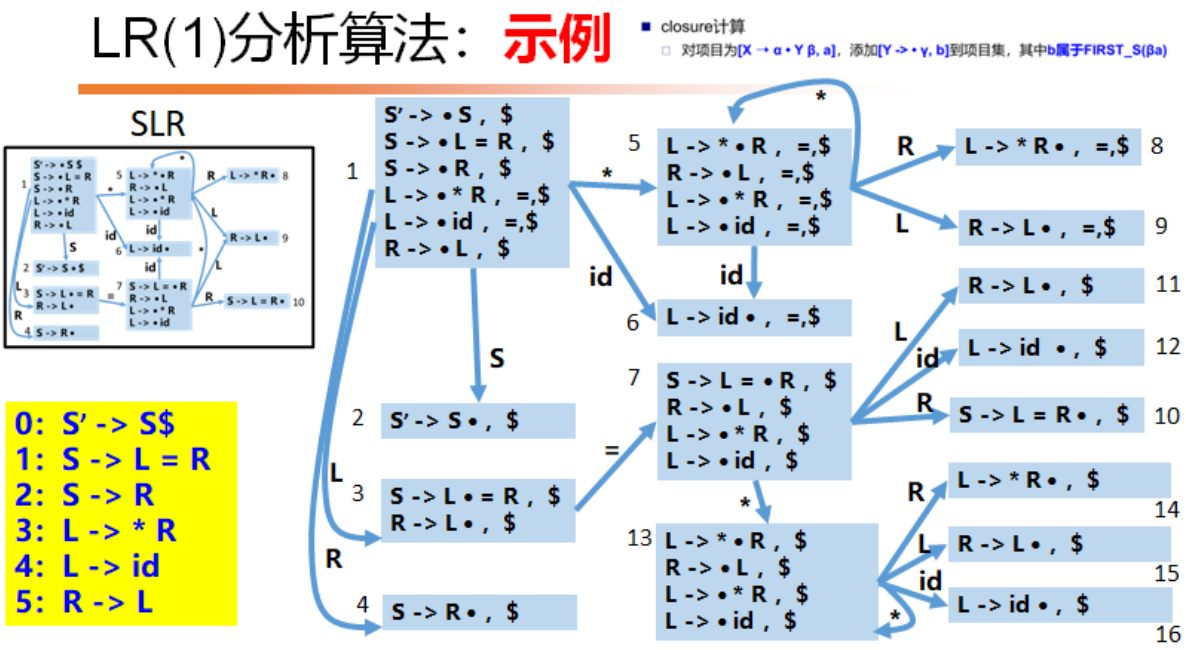
- LR(1)中定义项目为**[X → α • β, a]**，其含义是：
  - 已读入**α**
  - 剩余的输入能够匹配**βa**；若**β=ε**时，可以用该条产生式**归约**（α归约成X）
- LR(1)分析算法其他和LR(0)相同，仅**closure**(闭包)的计算和**归约表项**的产生不同
  - closure计算
    - 对项目为**[X → α • Y β, a]**，添加**[Y -> • γ, b]**到项目集，其中**b属于FIRST_S(βa)**
  - 归约表项的产生
    - 对包含项目**[X → α β •, a]**的状态，对**输入a**产生该产生式的**归约项rn**
    - **a是前看符号**

LR(1)分析算法：**示例**

- closure计算
  - 对项目为**[X → α • Y β, a]**，添加**[Y -> • γ, b]**到项目集，其中**b属于FIRST_S(βa)**



LR(1)分析算法：**示例**



	动作（ACTION）				转移（GOTO）		
	id	=	*	$	S	L	R
1	s6		s5		g2	g3	g4
2				accept			
3		s7		r5			
4				r2			
5	s6		s5			g9	g8
6		r4		r4			
7	s12		s13			g11	g10
8		r3		r3			
9		r5		r5			
10				r1			
11				r5			
12				r4			
13	s16		s13			g15	g14
14				r3			
15				r5			
16				r4			



注意，S'->S$改变处理方式了，改为S'->S，后跟$。

思考：LR1是否能杜绝冲突?

SLR做的是，左非终结符后面是可继续的，才归约。SLR的冲突是左非终结符后面是可继续的和该符号在·的后面同时存在。

LR1做的是，只有该产生式后面的是可继续的符号，才做归约。那么应该也可以，该产生式后面可继续，且产生式中有存在，如：

这个状态：

```
A->B·, c
D->B·c, e
```

这个语法就可以达到：

```
S'->S
S->Ac
A->De
A->B
D->Bc
B->f
```

状态1：

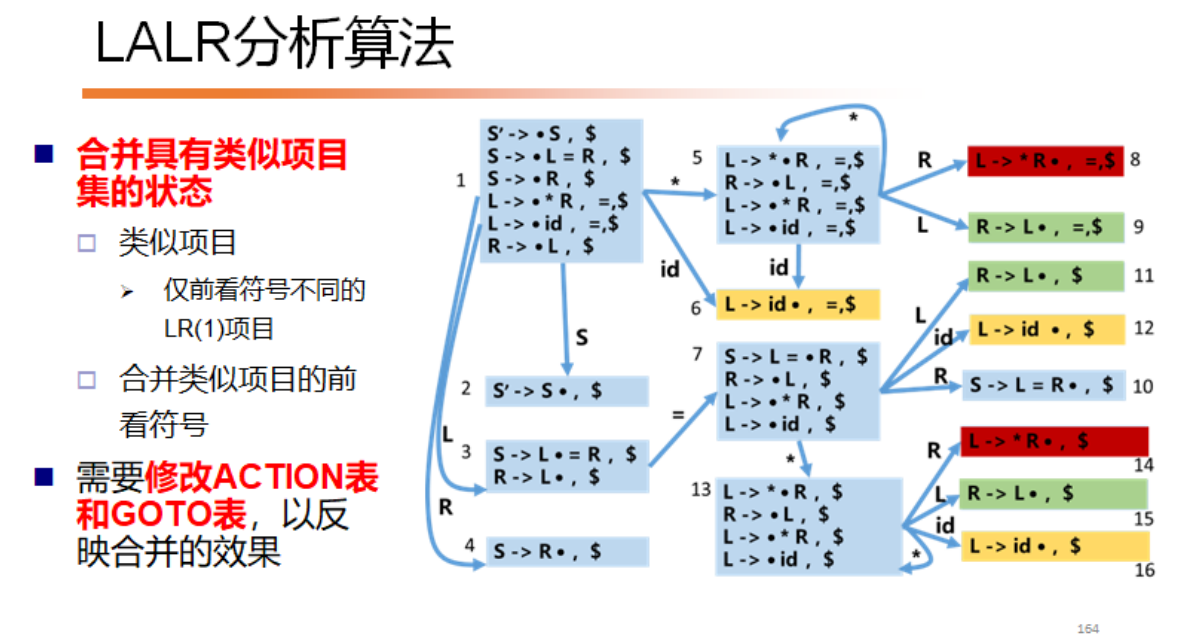
```
S'->·S,$
S->·Ac,$
A->·De,c
A->·B,c
D->·Bc,e
B->·f,c
```

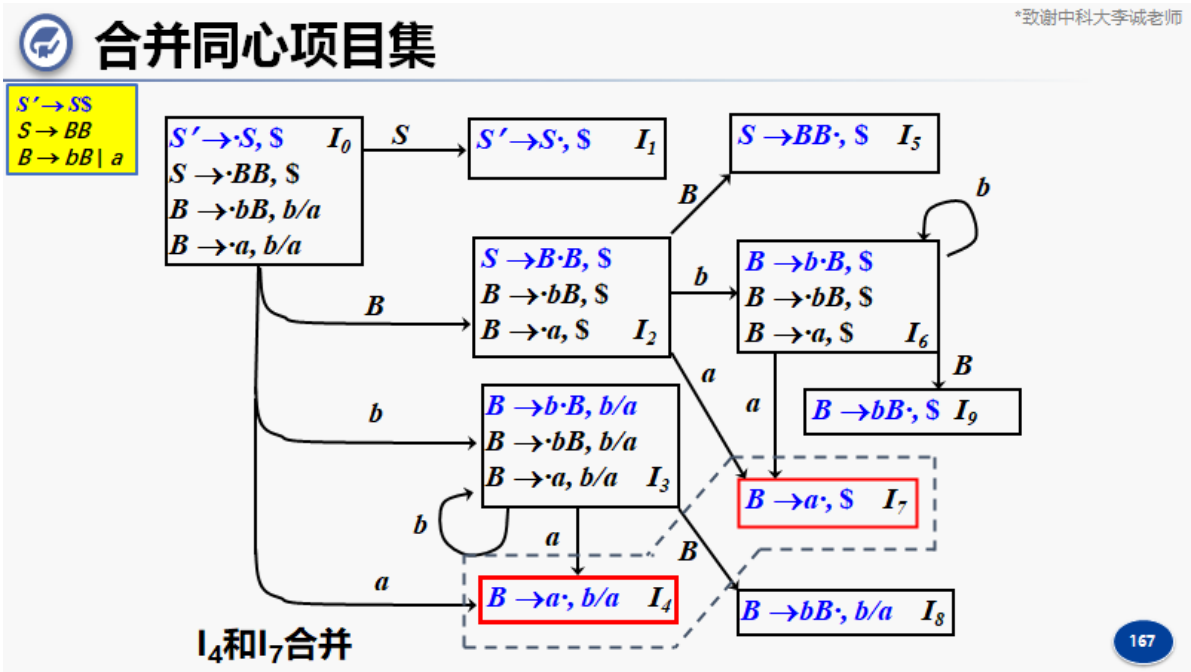
状态1通过B达到状态2：

```
A->B·,c
D->B·c,e
```

这个状态是有移进归约冲突的：当下一个是c，LR1就不知道是移进还是归约。

- LALR





思考：LALR是否会因为减少状态而增加出错可能

会的，应该是会的，合并后就相当于原来不应该的地方变应该了。

第四章：写SDD并执行，结合生成

- 语法制导翻译：用相应的语法规则时执行相应的翻译动作：

语法制导翻译用途：示例

本集提要!
This Episode!

翻译动作の初体验
算它!

```
1: E -> E+E     {E = E1 + E2}
2:    | n       {E = n}
```

现场演示
YACC



基本思想

语法分析时，给产生式规则附加一个翻译动作

```
1: X -> B1      A1
2:    | B2      A2
3:    | B3      A3
4:    | ...     ...
```

- 语法分析在**使用该条产生式的时候插入翻译动作**

示例

```
1: E -> E+E     {E = E1 + E2}
2:    | n       {E = n}
```

自顶向下分析

- 将**产生式展开成右部**时插入动作

```
int Parse_E()
{
  if(next_token()==n) {
    E1=n;
    E2=Parse_EPrime();
    return E1+E2
  }
}
```

```
int Parse_EPrime()
{
  if(next_token()==+) {
    return Parse_E();
  }
  else if(next_token()==epsilon) {
    return 0;
  }
}
```

消除左递归

```
E  -> nE'       {E = n+E'}
E' -> +EE'|ε    {E = E1|0}
```

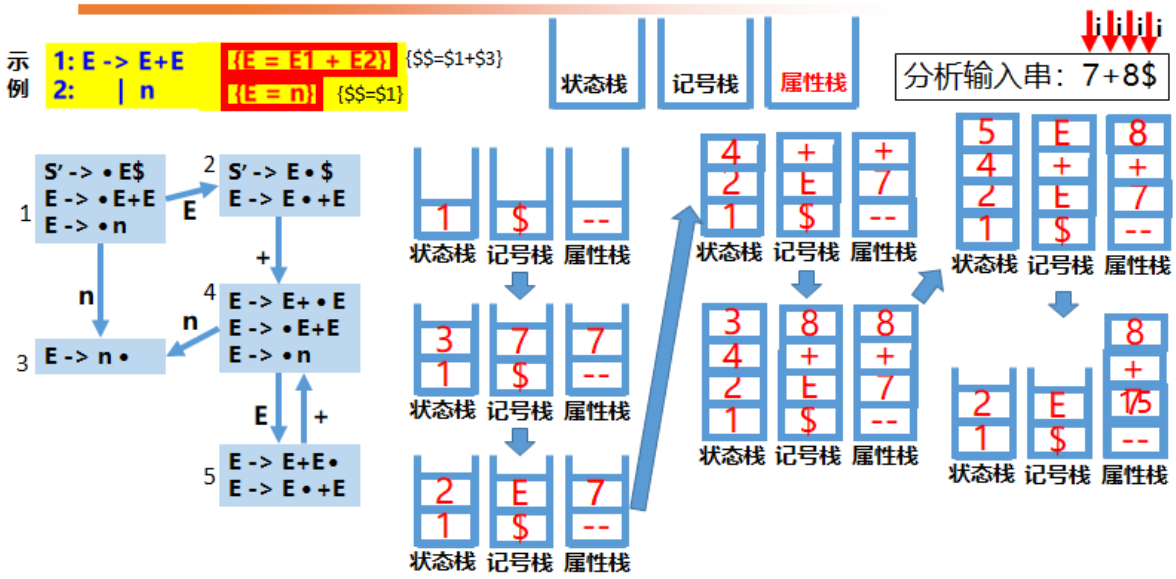
自底向上分析

- 将**产生式右部收缩成左部**时插入动作

```
if (ACTION[s, t] == "rn")  "n: X -> β1 … βj"
    if(rn==r1)
      X=β1+β3
    if(rn==r2)
      X=β1
    pop(token_stack);
    pop(state_stack);
    push(token_stack, X);
    push(state_stack, GOTO[s,X]);
```


（以下ppt中有演示，也就那样没什么难度）

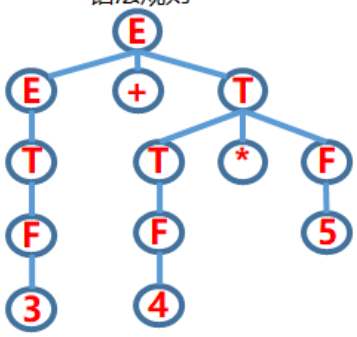
语法制导翻译的具体实现：示例



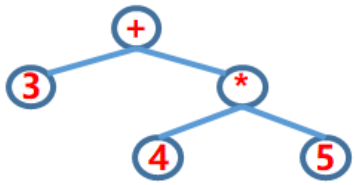
- 抽象语法树：通过语法制导翻译实现，可人为设计翻译规则

语法分析树 V.S. 抽象语法树

- 语法分析树（Parse Tree）
  - 对应**具体语法**
    - **具体语法**是语法分析器使用的语法，描述了语言的具体语法规则



- 抽象语法树（Abstract Syntax Tree）
  - 对应**抽象语法**
    - **抽象语法**是编译器用来描述词法语法分析过后的输入程序的内部数据结构，供后续编译阶段使用

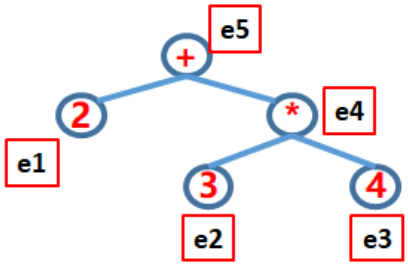


抽象语法树的具体数据结构

- 来看一个C语言版的抽象语法树数据结构定义
  - 示例：用数据结构来编码程序“2+3*4”

```
E -> n
   | E+E
   | E*E
```

```
e1 = Exp_Int_new (2);
e2 = Exp_Int_new (3);
e3 = Exp_Int_new (4);
e4 = Exp_Times_new (e2, e3);
e5 = Exp_Add_new (e1, e4);
```



（下图，左边是抽象语法树。这是通过语法制导翻译实现的，在合适的地方翻译创建节点和边）

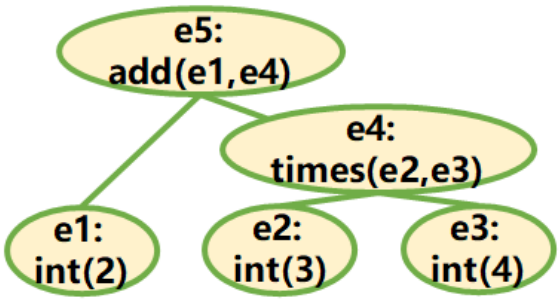
如何自动生成抽象语法树?

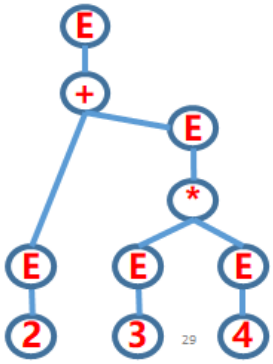
- 以LR语法分析中生成抽象语法树为例
  - 在产生式归约的时候，会自底向上构造整棵树（从叶子到根）

```
E -> n      {e=Exp_Int_new(n)}
   | E+E    {e=Exp_Add_new(e1,e2)}
   | E*E    {e=Exp_Times_new(e1,e2)}
```

输入语句 **2+3*4**

- 2 • + 3 * 4
- E • + 3 * 4
- E + 3 • * 4
- E + E • * 4
- E + E * 4 •
- E + E * E •
- E + E •
- E •





源代码信息的保留和传播

- 抽象语法树是编译器前端和后端的接口
  - 程序一旦被转换成抽象语法树，则源代码即被丢弃
  - 后续的阶段只处理抽象语法树
- 所以抽象语法树必须编码足够多的源代码信息
  - 例如，它必须编码每个语法结构在源代码中的位置（文件、行号、列号等）
    - 这样，后续的检查阶段才能精确的报错
    - …
- 抽象语法树必须仔细设计！



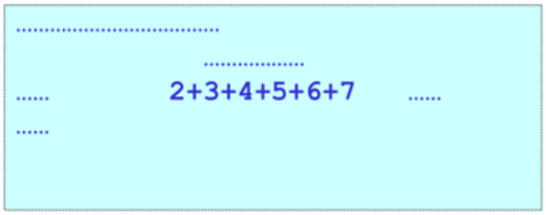
30

源代码信息的保留和传播

- 示例：添加位置信息

```
struct position_t{
  char *file;
  int line;
  int column;
};

struct Exp_Add{
  enum kind kind;
  Exp *left;
  Exp *right;
  struct position_t from;
  struct position_t to;
};
```

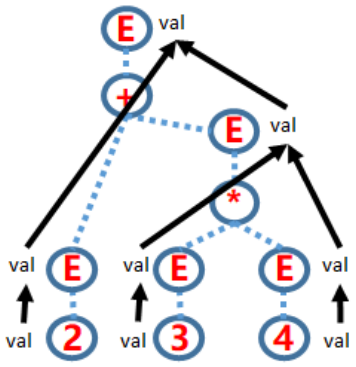


31

- 理论上，可以进行SDD（语法制导翻译）的，必须要满足以下条件之一。实际上就是说“不能递归就好”：

怎样在语法分析过程中搞定这些SDD的翻译?

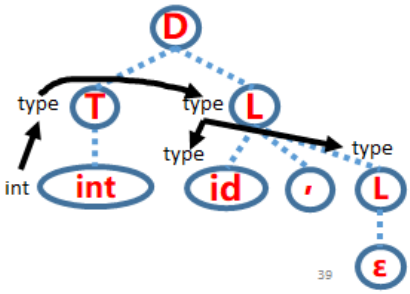
- **S属性的SDD**
  - 每个属性都是综合属性
  - 即，只存在**自底向上**的属性依赖关系
- 自底向上语法分析过程中
  - 在产生式**右部规约为左部时执行翻译动作**
  - So easy!



怎样在语法分析过程中搞定这些SDD的翻译?

- **L属性的SDD**
  - 只存在**自底向上/自顶向下/自左向右**的属性依赖关系
- 递归下降语法分析过程中
  - 构造非终结符N的子分析函数时
    - N的**继承属性**是子函数的**参数**
    - N的**综合属性**是子函数的**返回值**
    - 左边的子函数先执行，所以左依赖的属性值先计算

D -> T L	{L.type=T.type}
T -> int	{T.type=int}
T -> float	{T.type=float}
L -> id , L_1	{id.type=L.type; L_1.type=L.type}
L -> ε	




```
Parse_D() {
  t.type=Parse_T();
  l.type=t.type;
  Parse_L(l.type);
}
```

```
Parse_L(type t) {
  if(next_token()==id) {
    id.type=t;
    Parse_L(t);
  } else if(next_token()==eps) {
```


第五章：到哪步做什么事情

- 语义分析

语义分析概述

- **语义分析（Semantic Analysis）**也称为类型检查、上下文相关分析

抽象语法树

- 负责检查程序的**上下文相关的属性**
  - 变量在使用前先进行声明
  - 每个表达式都有合适的类型
  - 函数调用和函数的定义一致
  - 。。。

与具体语言相关

语义分析示例

- C 代码。有哪些错误?

```
void f (int *p)
{
    x += 4;
    p (23);
    “hello” + “world”;
}
int main ()
{
    f () + 5;
    break;
    return;
}
```

- 这些语义错误的检查依赖上下文信息，**语法分析搞不定**!
  - 例如类型检查

```
P -> D E
D -> T id; D
   | ε
T -> int
   | bool
E -> n
   | id
   | true
   | false
   | E + E
   | E += E
   | E && E
```

该产生式对不对取决于前面T申明了什么类型

程序语言的语义规则

- 目前为止我们学到的编译各阶段用到的规则的描述方法

词法	语法	语义
正则表达式☺	**上下无关文法☺**	***自然语言☹***

- 通常来说，程序设计语言都采用**自然语言**来表达程序语言的语义
  - 例如，对于”+” 运算，要求左右操作数都必须是整型数
- **与具体语言相关**
  - 编译器的实现者必须对语言中的语义规定有全面的理解
    - 主要的挑战是如何能够正确高效的实现

C--语言的语义规则

- 以C--语言的典型语义规则为例，讨论如何实现语义分析
  - 每个表达式都有合适的类型
    - “+”左右只能是整型
    - “&&”左右只能是布尔类型
  - 变量在使用前先进行声明

```
//语义合法的程序：
3+4
false && true
//语义不合法的程序：
3 + true
true + false
```

```
// 语义合法的程序：
int x;
x+4
//语义不合法的程序：
x + 3
```


（可以看到，以下语义分析，为了判断类型匹配，就先获取了相应的类型再判断就好了，非常常规的操作）

语义1：每个表达式都有合适的类型

- 对给定的一个表达式E，写一个函数check(E)返回表达式E的类型；若类型不合法，则报错
  - E为语法分析过后生成的表达式E的抽象语法树数据结构

```
E -> n
   | true
   | false
   | E + E
   | E && E
```

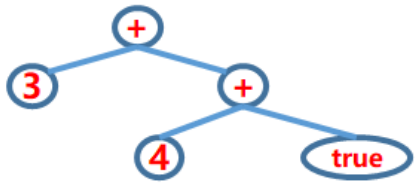
```
enum type {INT, BOOL};

enum type check_exp (Exp_t e) {
  switch(e->kind)
    case EXP_INT: return INT;
    case EXP_TRUE: return BOOL;
    case EXP_FALSE: return BOOL;
    case EXP_ADD:
      t1 = check_exp (e->left);
      t2 = check_exp (e->right);
      if (t1!=INT || t2!=INT)
        error (“type mismatch”);
      else return INT;
    case EXP_AND: … //类似
}
```

示例1



示例2

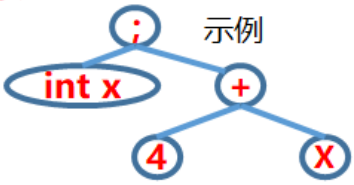


（以下代码不完整，不好推断。猜测也就是直接遍历查表就是了，非常常规）

语义2：变量在使用前先进行声明

- 需要用到辅助数据结构：**符号表**

```
P -> D E
D -> T id; D
   | ε
T -> int
   | bool
E -> n
   | id
   | true
   | false
   | E + E
   | E && E
```




```
enum type {INT, BOOL};
Table_t table;

void check_prog (Dec_t d, Exp_t e) {
  table = check_dec (d)
  check_exp (e)
}

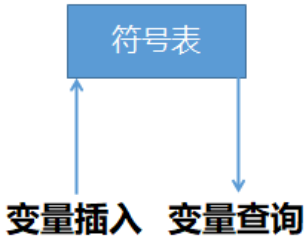
Table_t check_dec (Dec_t d) {
  foreach (T id belongs to d)
    table_enter (table, id, T)
  return table
}
```

```
enum type check_exp (Exp_t e){
  switch (e->kind)
    case EXP_ID:
      t = Table_lookup (table, id)
      if (id not exist)
      error ("id not found")
      else return t
}
```

15

- 符号表（一个组件）

符号表的接口




```
// 符号表的插入
void Table_enter (Table_t, Key_t, Value_t);
// 符号表的查找
Value_t Table_lookup (Table_t, Key_t);
```

18

符号表的典型数据结构

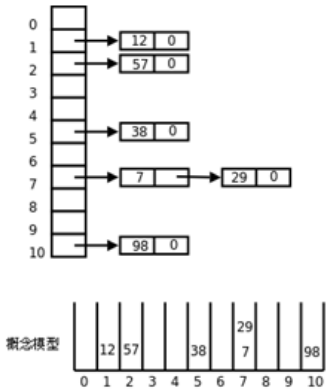
- 符号表是典型的**字典结构**
  - symbolTable: key -> value
- 一种简单的数据结构的定义

```
typedef char *key;
typedef struct value{
    Type_t type;
    Scope_t scope;
    ... // 必要的其他字段
} value;
```

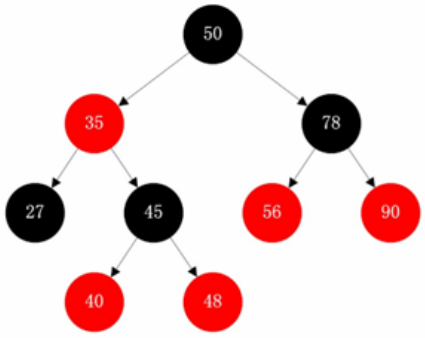
键\值	type	scope	...
x	INT	0	...
y	BOOL	1	...
...	...	...	...

符号表的一般高效实现

- 考虑**速度**，可以使用**哈希表**等数据结构来实现符号表
  - 时间是O(1)，空间是O(K)



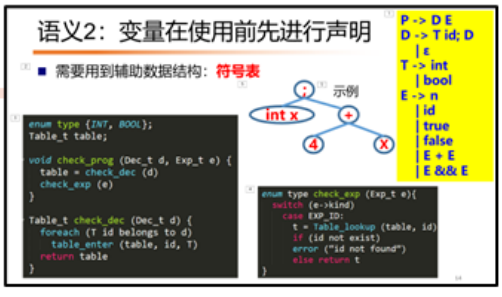
- 考虑**空间**，也可以使用**红黑树**等平衡树
  - 时间是O(lg N)，空间是O(N)



简单情况示例

举个栗子





// 语义合法的程序：

```
int x;
bool y;
x+4
```

INT

键\值	type	scope
x	INT	0
y	BOOL	0

//语义不合法的程序：

```
bool x;
x+ 3
```

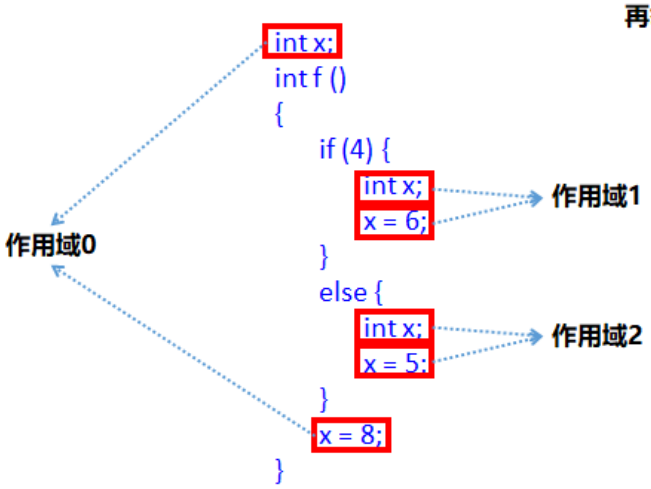
BOOL

键\值	type	scope
x	BOOL	0

复杂情况示例：作用域



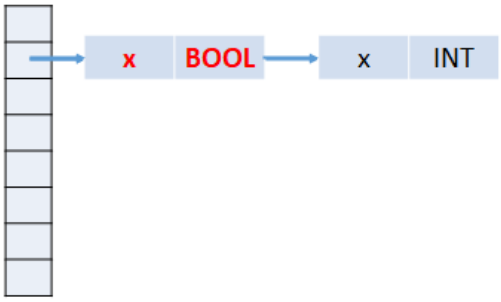
再举个栗子



复杂情况示例：作用域

```
int x;
int f ()
{
    if (4) {
        float x;
        x = 6.1;
    }
    else {
        bool x;
        x = true;
    }
    x = 8;
}
```

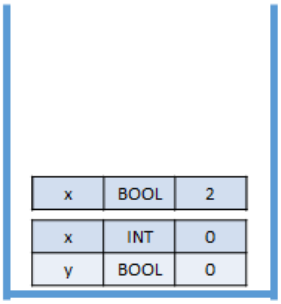
- 方法#1：一张表的方法
  - **进入**作用域时，**插入**元素
  - **退出**作用域时，**删除**元素



复杂情况示例：作用域

```
int x;
int f ()
{
    if (4) {
        float x;
        x = 6.1;
    }
    else {
        bool x;
        x = true;
    }
    x = 8;
}
```

- 方法#2：采用符号表构成的**栈**
  - **进入**作用域时，**插入**新的符号表
  - **退出**作用域时，**删除**栈顶符号表



复杂情况示例：名字空间



再再举个栗子

```
struct list
{
    int x;
    struct list *list;
} *list;

void walk (struct list *list)
{
    list:
        printf ("%d\n", list->x);
        if (list = list->list)
            goto list;
}
```

不同的名字空间:
① 名字
② 变量
③ 成员
④ 标号

- **每个名字空间用一个符号表来处理!**

名字表

list	STRUCT	0

成员表

list	STRUCT list	0

变量表

list	list	0

标号表

list	Line XXX	0

25

也并不难懂

- 类型相容（虽然类型不相同，但是还是可以定义运算，如int+bool先把bool转为int。这些都是工程问题。）

类型相容是指编程语言中，两种不同的数据类型在进行某些操作时能够自动转换为相同的类型或者直接进行操作而无需转换的特性。（chatgpt）

类型相容

- 问题：a = b 是否符合语义规则
  - 如果a和b的类型**相同**，符合
  - 如果a和b的类型**相容**，也符合
- 示例1：**名字相等vs结构相等**
  - 对采用名字相等的语言，可直接比较
  - 对采用结构相等的语言，需要递归比较各个域

```
struct A
{
  int i;
} x;
struct B
{
  int i;
} y;
x = y;
```

键\值	type
x	A
y	B

键\值	field
A	int i
B	int i

28

类型相容

- 问题：a = b 是否符合语义规则
  - 如果a和b的类型**相同**，符合
  - 如果a和b的类型**相容**，也符合
- 示例1：**面向对象的继承**
  - 需要维护类型间的继承关系

```
class A
{
  int i;
}
class B: A
{
  int i;
}
A x;
B y;
x = y;
```

键\值	type
x	A
y	B

键\值	field	parent
A	int i	
B	int i	A

29

- 扩展讨论

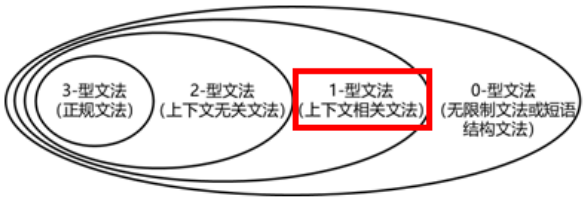
来点彩蛋



词法
正则表达式☺

语法
上下无关文法☺

语义
自然语言☹



乔姆斯基文法体系

为什么不用上下文相关文法这个数学工具?

- 上下文相关文法分析**时间复杂度高**!
  - NP-complete
- 实际程序的语义规则用上下文相关文法**描述非常复杂**!
  - $\alpha A\beta \rightarrow \alpha\gamma\beta$

31

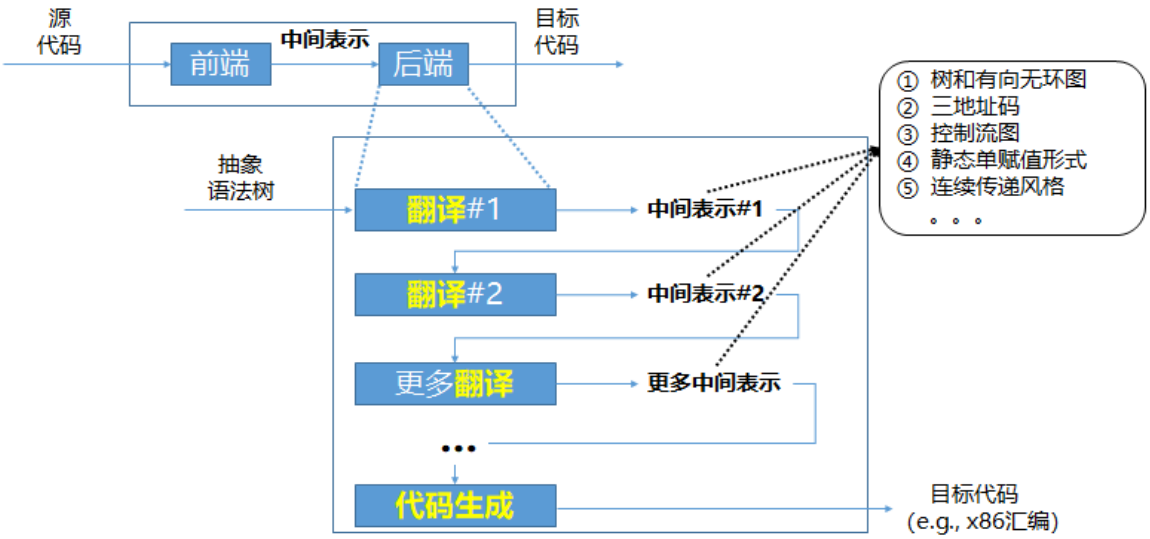
我们用的不是上下文相关文法，用的是等价的直接工程实现，抛弃了形式化统一的繁琐。

第六章：3地址码，和第七章中间表示一起

- 代码生成

属于到后端部分，目的就不是报错和简化，而是生成代码了。现代也经常和语义分析一起进行。

编译器**后端**



代码生成概述

- 负责把源程序翻译成“**目标机器**”上的代码
  - 目标机器：
    - 可以是**物理机**
      - x86, ARM, MIPS, RISC-V…
    - 可以是**虚拟机**
      - JVM…
- 两个任务：
  - 给源程序的**数据**分配**存储资源**
  - 给源程序的**代码**选择**指令**

首要目标：
等价！
次要目标：
效率！

给源程序的数据分配存储资源

- 源程序的**数据**：
  - 全局变量、局部变量、动态分配等
- 机器**存储资源**：
  - 寄存器、内存
- 根据程序的特点和编译器的设计目标，合理分配存储资源
  - 例如：变量放在内存里还是寄存器里？

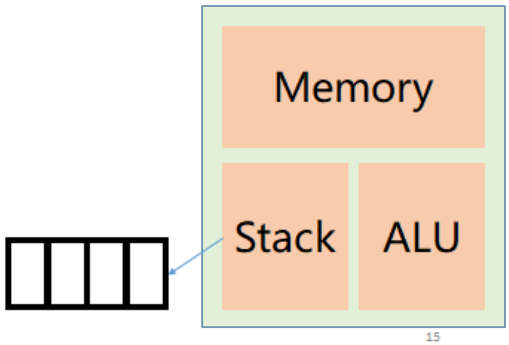
给代码选择合适的机器指令

- 源程序的**代码**：
  - 表达式运算、语句、函数等
- 机器**指令**：
  - 算术运算、比较、跳转、函数调用返回
- 用机器指令实现高层代码的语义
  - 对机器**指令集体系结构**（ISA）的熟悉

- 示例：栈式计算机

栈式计算机Stack的结构

- 内存 (Memory)
  - 存放所有的**变量**
- 算术逻辑单元 (ALU)
  - 执行运算**指令**
- 栈 (Stack)
  - 进行**运算的空间**
    - 运算操作数的存放
    - 暂存运算的中间结果
    - 。。。



栈计算机的指令集

```
push  NUM
load  x
store x
add
sub
times
div
```

栈操作

访存

算术运算

与JAVA字节码相似，是其子集

变量的内存分配伪指令

- Stack机器只支持一种数据类型int，并且给变量x分配内存的伪指令是:
  - .int x
- Stack机器在装载一个程序时，就会读取伪指令，并且给相关变量分配内存空间

示例

```
int x;
int y;
int z;

x = 10;
y = 5;
z = x + y;
y = z * x;
```

源代码

```
.int x
.int y
.int z

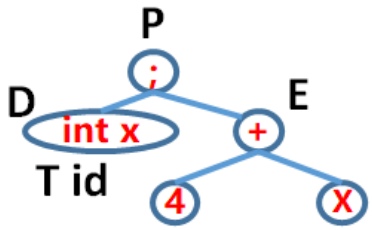
push 10
store x
push 5
store y
load x
load y
add
store z
load z
load x
times
store y
```

栈式计算机上的目标代码

栈式计算机代码生成思路

- **递归下降**的代码生成方法！

```
P -> D S
D -> T id; D
   | ε
T -> int
   | bool
S -> id = E; S
   | ε
E -> n
   | id
   | true
   | false
   | E + E
```



抽象语法树

```
Gen_P( );
Gen_D( );
Gen_T( );
Gen_S( );
Gen_E( );
```

每种类型的语法树节点构造一个代码生成函数

22

（下图代码一部分被挡了，看ppt有演示。大意就是Gen_X是对于X的自定义操作，其中emit打印出了对应目标代码）

（D是声明部分，S是运算部分，使用递归方法。）

栈式计算机代码生成：示例

```
P -> D S
D -> T id; D
   | ε
T -> int
   | bool
S -> id = E; S
   | ε
E -> n
   | id
   | true
   | false
   | E + E
```

```
Gen_P(D S) {
  Gen_D (D);
  Gen_S (S);
Gen_S(S s) {
  switch (s)
  case id=e:
      Gen_E(e);
      emit("store id");
      Gen_S(S s);
  case epsilon:
Gen_E(E e) {
  switch (e)
    case n: emit ("push n"); break;
    case id: emit ("load id"); brea
    case true: emit ("push 1"); break;
    case false: emit ("push 0"); break;
    case e1+e2: Gen_E (e1);
                Gen_E (e2);
                emit ("add");
                break;
    case …: … // similar
}
```

```
int x;
int y;
int z;

x = 10;
y = 5;
z = x + y;
y = z * x;
```

源代码

```
.int x
.int y
.int z

push 10
store x
push 5
store y
load x
load y
add
store z
load z
load x
times
store y
```

栈式计算机上的目标代码

27

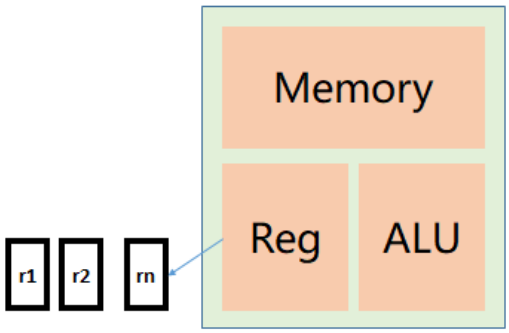
- 示例：寄存器计算机

寄存器计算机概览

- 寄存器计算机是目前最流行的机器体系结构之一
  - 效率很高
  - 机器体系结构规整
- 机器基于寄存器架构：
  - 典型的有16、32或更多个寄存器
    - 所有操作都在寄存器中进行
  - 访存都通过load/store进行
    - 内存不能直接运算

寄存器计算机Reg的结构

- 内存 (Memory)
  - 存放**溢出**的**变量**
- 算术逻辑单元 (ALU)
  - 执行运算**指令**
- 寄存器 (Register)
  - 进行**运算的空间**
    - 存放部分变量
    - 运算操作数的存放
    - 暂存运算的中间结果
    - 。。。



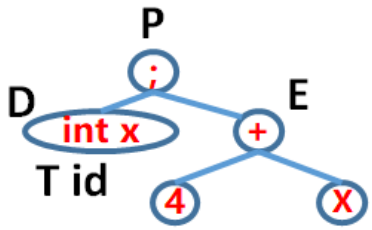
变量的寄存器分配伪指令

- Reg机器只支持一种数据类型int，并且给变量x**分配寄存器**的伪指令是：
  - **.int x**
- 在目前的代码生成阶段，假设Reg机器上有**无限多个寄存器**
  - 因此每个声明变量和临时变量都会占用一个（虚拟）寄存器
  - 把虚拟寄存器分配到物理寄存器的过程称为**寄存器分配**

寄存器计算机Reg代码生成思路

- **递归下降**的代码生成方法！

```
P -> D S
D -> T id; D
   | ε
T -> int
   | bool
S -> id = E; S
   | ε
E -> n
   | id
   | true
   | false
   | E + E
```



抽象语法树

```
Gen_P( );
Gen_D( );
Gen_T( );
Gen_S( );
Gen_E( );
```

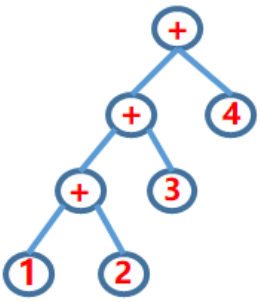
每种类型的语法树节点构造一个代码生成函数

33

（代码在前几页ppt，和栈计算机差不多的，就是针对寄存器的不同工程实现）

寄存器计算机代码生成：示例

```
P -> D S
D -> T id; D
   | ε
T -> int
   | bool
S -> id = E; S
   | ε
E -> n
   | id
   | true
   | false
   | E + E
```



“1+2+3+4”的抽象语法树

源代码

```
int x;
int y;
int z;

x = 1+2+3+4;
y = 5;
z = x + y;
```

```
.int x
.int y
.int z

movn 1, r1
movn 2, r2
add r1, r2, r3
movn 3, r4
add r3, r4, r5
movn 4, r6
add r5, r6, r7
store r7, x
movn 5, r8
store r8, y
load x, r9
load y, r10
add r9, r10, r11
store r11, z
```

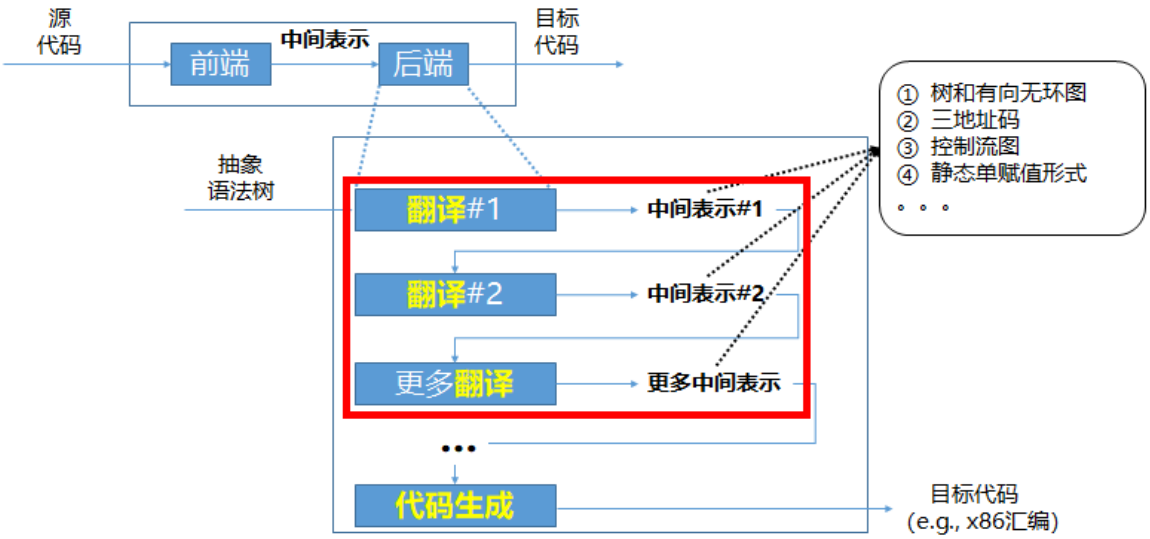
栈式计算机上的目标代码

第七章：3地址码，控制流图，有向无环图优化等

- 中间表示

要反复优化中间表示。

真实的编译器**后端**



中间表示概览

- 中间表示（Intermediate Representation，IR）
  - 编译器内部使用的一种**抽象语言**
    - 表示**目标机**的操作
    - 包含**源语言**的信息
- 基本思想
  - 与**目标机器无关**
  - 对于多种**源语言通用**

为什么需要**多阶段**中间表示?

- 编译器**工程**上的考虑
  - **阶段划分**：把整个编译过程划分成不同的阶段
  - **任务分解**：每个阶段只处理翻译过程的一个步骤
  - **软件工程**：代码更容易实现、除错、维护和演进
- 代码**优化**的需要
  - 和程序的中间表示密切相关
    - 许多优化在特定的中间表示上才可以或才容易进行

一般编译器的多阶段中间表示

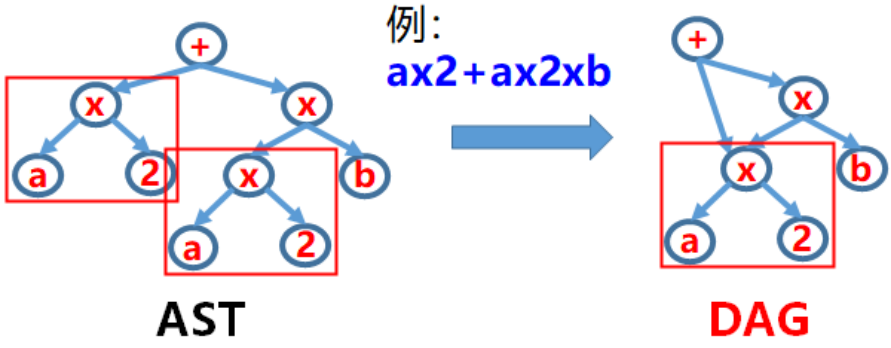


- 由 **高层**中间表示 到 **低层**中间表示
  - 高层中间表示接近**源语言**
    - 适用面向源码的程序分析和优化
      - 语义分析、冗余检查。。。
  - 低层中间表示接近**目标语言**
    - 适用面向目标语言的优化和翻译
      - 寄存器分配、常量优化。。。

- 示例：有向无环图表示

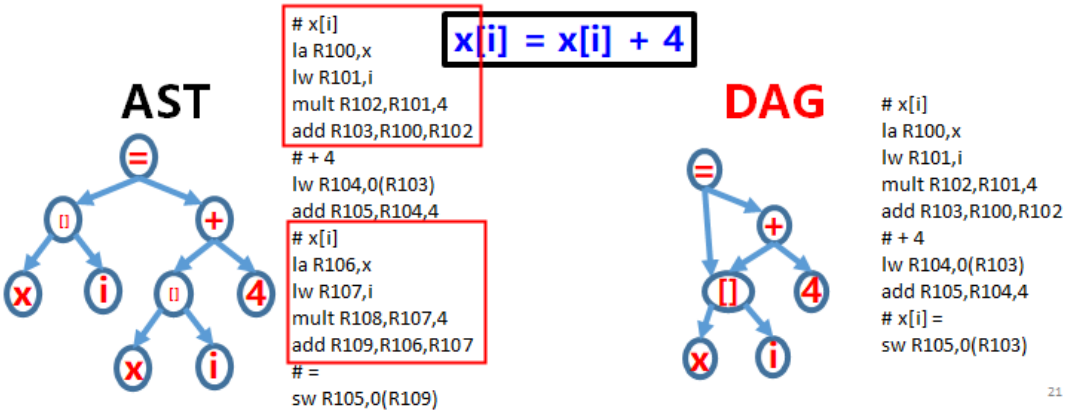
有向无环图

- 有向无环图（Directed Acyclic Graph，DAG）
  - DAG是一种具有共享机制的AST。**相同的子树只实例化一次，可能有多个父节点。**



有向无环图的用途

- 实际系统中使用DAG主要有两个原因
  - **减少内存占用**
    - 比如R[n]语言的AST内存开销太大
  - **后端代码优化**
    - 查找代码的冗余之处，减少翻译出的机器指令



21

有向无环图的生成

- 生成抽象语法树节点时
  - **查询是否已生成，如果已生成则直接引用**

	PRODUCTION	SEMANTIC RULES
1)	$E \to E_1 + T$	$E.node =$ **new** $Node('+', E_1.node, T.node)$
2)	$E \to E_1 - T$	$E.node =$ **new** $Node('-', E_1.node, T.node)$
3)	$E \to T$	$E.node = T.node$
4)	$T \to (\ E\ )$	$T.node = E.node$
5)	$T \to$ **id**	$T.node =$ **new** $Leaf(\text{id}, \text{id}.entry)$
6)	$T \to$ **num**	$T.node =$ **new** $Leaf(\text{num}, \text{num}.val)$

```
If node/leaf p exits
   return p;
else
   new p and return;
```

AST的语法制导翻译

DAG的节点创建过程不同

23

- 示例：三地址码

三地址代码

- 三地址码（three address code）
  - 一条指令最多有三个地址（操作数）、一个运算符
  - 主要形式：**x = y op z** 或 **if (x relop y) goto L**
- 常见的三地址码形式

```
x = y ⊕ z          // 二元运算
x = ⊖ y            // 一元运算
x = y              // 数据移动
x[y] = z           // 内存写
x = y[v]           // 内存读
x = call f, n      // 函数调用
goto L             // 无条件跳转
if (x relop y) goto L  // 条件跳转
label L            // 标号
y = call p, n      // 函数调用
```

例：

```
z ← x - 2 * y
```

源码

```
t ← 2 * y
z ← x - t
```

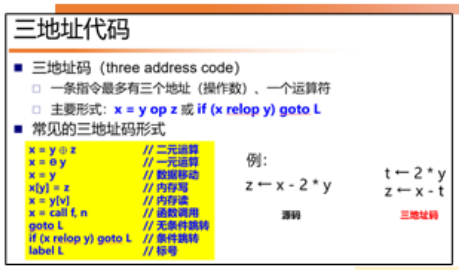
三地址码

三地址码的基本思想

- 给每个中间变量和计算结果命名
  - 没有复合表达式
- 只有最基本的控制流
  - 没有各种控制结构
  - 只有goto，call等
- 三地址码可以看成是**抽象的指令集**
  - 类似通用的RISC

```
x = y ⊕ z          // 二元运算
x = ⊖ y            // 一元运算
x = y              // 数据移动
x[y] = z           // 内存写
x = y[v]           // 内存读
x = call f, n      // 函数调用
goto L             // 无条件跳转
if (x relop y) goto L  // 条件跳转
label L            // 标号
y = call p, n      // 函数调用
```


示例




```
a = 3 + 4 * 5;

if (x < y)
  z = 6;
else
  z = 7;
```

源码

```
t1 = 4 * 5;
a = 3 + t1;
if (x < y) goto L_1;
goto L_2;
L_1:
z = 6;
goto L_3;
L_2:
z = 7;
L_3:
...
```

三地址码

三地址码的用途

- 类似大多数目标机器语言，便于**代码生成**
  - 类似寄存器计算机
- 代码紧凑，**空间占用少**
  - 一个操作、三个地址
    - 操作和名字都取自有限集合，一般4个字节足够表示三地址码
- 形式灵活，表达清晰，**优化空间大**
  - 给每个中间变量和计算结果命名
    - 便于控制名字和值的复用

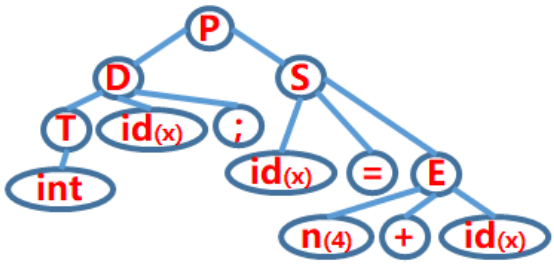
三地址码的生成方法

- **递归下降**的代码生成方法！
  - 例：为C-语言生成三地址码

```
P -> D S
D -> T id; D
   | ε
T -> int
   | bool
S -> id = E
   | if (E, S*, S*)
   | while (E, S*)
E -> n
   | id
   | true
   | false
   | E op E
   | E relop E
```

```
Gen_P( );
Gen_D( );
Gen_T( );
Gen_S( );
Gen_E( );
```

每种类型的语法树节点构造一个代码生成函数



语法分析树
例：int x; x=4+x

（也就是个工程实现。）

三地址码小结

- 三地址码的**优点**：
  - 所有的操作是原子的
    - 变量！没有复合结构
  - 控制流结构被简化了
    - 只有跳转
  - 是抽象的机器代码
    - 向后做代码生成更容易
- 三地址码的**不足**：
  - 程序的控制流信息是隐式的
  - 可以做进一步的控制流分析

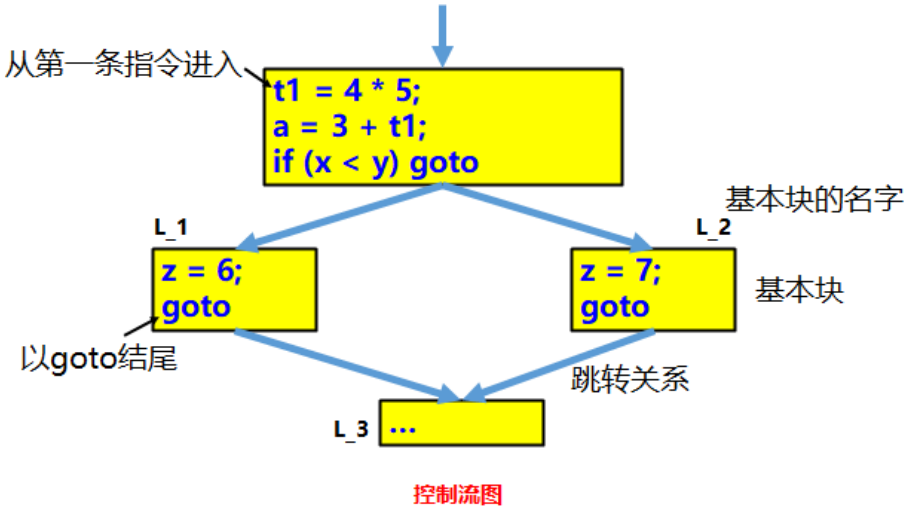
- 示例：控制流图

控制流图基本概念

- 控制流图（Control Flow Graph，CFG）

- **基本块**：一块连续指令，从第一条执行到最后一条
  - 只能从第一条语句进入该基本块
    - 不能从中间进入
  - 只能从最后第一条语句离开该基本块
    - 不能从中间退出，跳转指令只能出现在最后
- **控制流图**：控制流图是一个有向图G=(V，E)
  - 节点V：是基本块
  - 边E：是基本块之间的跳转关系
    - 指明哪些基本块可能紧随哪些基本块之后运行

示例



控制流图

控制流图的用途

- 程序的控制流图表示带来很多好处:
  - **控制流分析**:
    - 对很多程序分析来说，程序的内部结构很重要
    - 典型的问题："程序中是否存在循环？"
  - 可以进一步进行其他分析:
    - 例如**数据流分析**
      - 典型的问题："程序第5行的变量x可能的值是什么？"

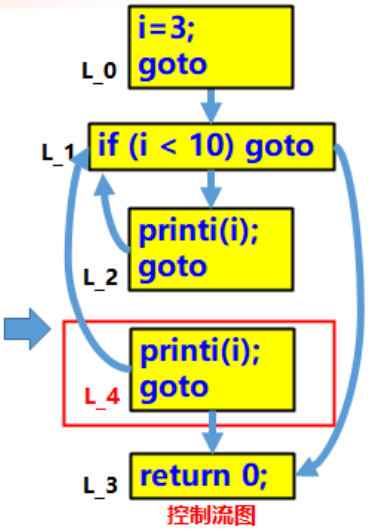
用途示例：删除死基本块

```
int f ()
{
  int i = 3;
  while (i<10) {
    printi(i);
    continue;
    printi(i);
  }
  return 0;
}
```

源码

```
L0:
i=3;
L1:
if (i < 10) goto L_2;
goto L_3;
L_2:
printi(i);
goto L_1;
printi(i);
goto L_1;
L_3:
return 0;
```

三地址码



控制流图

由三地址码生成控制流图算法

```
next ← 1
Leader[next++] ← 1 // 第一个基本块的开始为代码首行
for i ← 1 to n {// n为代码总行数
  if(op i has a label li) {
    Leader[next++] ← i
    create a CFG node li
  }
}
```

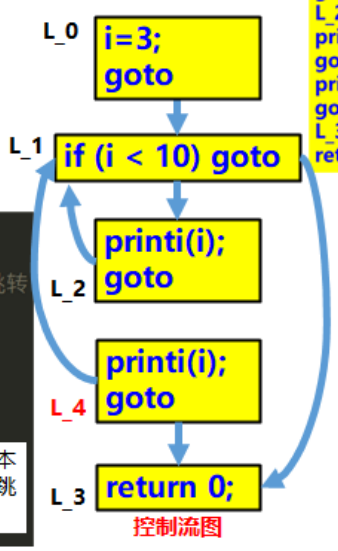
找出所有基本块的开始

```
for i ← 1 to next - 1
  Last[i] ← Leader[i+1]-1 // 获取基本块的最后一行指令
  if(Last[i] is "if e goto l1; goto l2;") { // 条件跳转
    add edge from node li to node l1
    add edge from node li to node l2
  }
  else if(Last[i] is "goto l1") { // 无条件跳转
    add edge from node li to node l1
  }
  else { // 没有跳转指令，默认跳转至下一个基本块
    add edges from node li to node li+1
  }
```

产生基本块间的跳转边

```
L0:
i=3;
L1:
if (i < 10) goto L_2;
goto L_3;
L_2:
printi(i);
goto L_1;
printi(i);
goto L_1;
L_3:
return 0;
```

三地址码

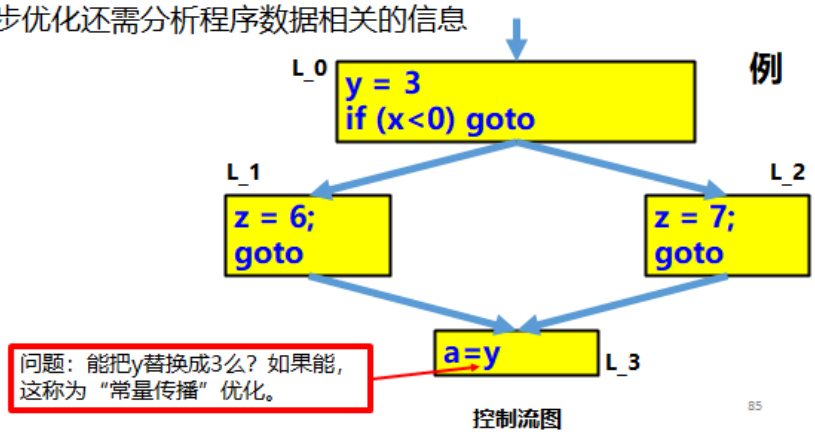


控制流图

- 示例：静态单赋值形式

控制流图的不足

- 只包含控制流信息，没有显示的**数据流**信息
  - 仅有基本块间的跳转关系还不够
  - 代码的进一步优化还需分析程序数据相关的信息



控制流图

静态单赋值形式

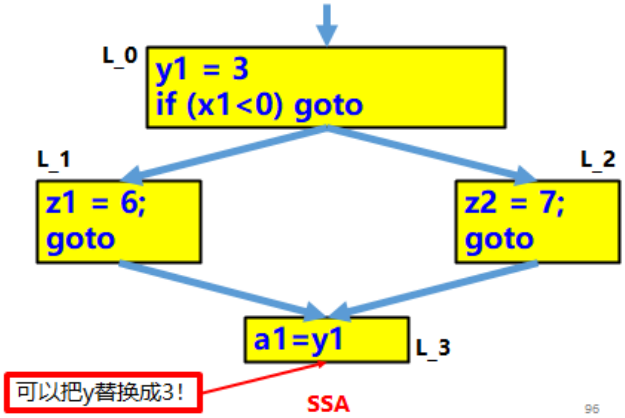
- 静态单赋值形式（Static Single Assignment Form，SSA）
  - 同时编码**控制流**信息和**数据流**信息
    - 更精细的控制流图
- 基本思想：在控制流图的基础之上
  - 每个名字（变量）只能定义一次
  - 使用时必须指向特定的一个名字（变量）
    - 引入**φ函数**，从多个变量中选择唯一的一个变量

SSA的用途

- 基于控制流和数据流的优化，如
  - 删除冗余赋值
  - 常量传播

```
y = 3;
if (x < y)
  z = 6;
else
  z = 7;
a=y
```

源码

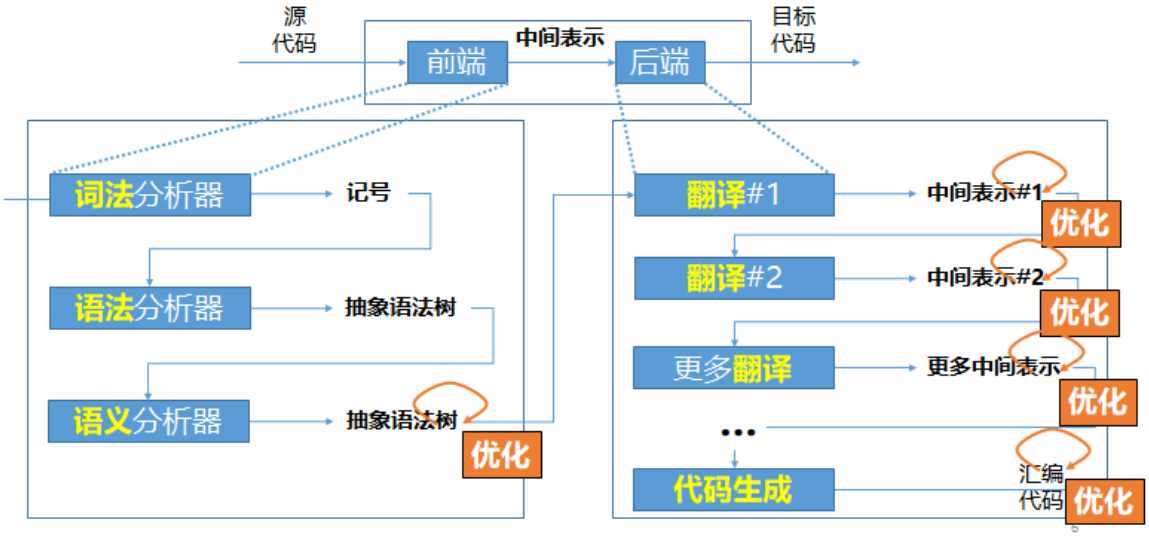


SSA

第八章：中期到达定义，活跃变量；后期寄存器分配

- 代码优化

代码优化的位置



什么是代码优化？

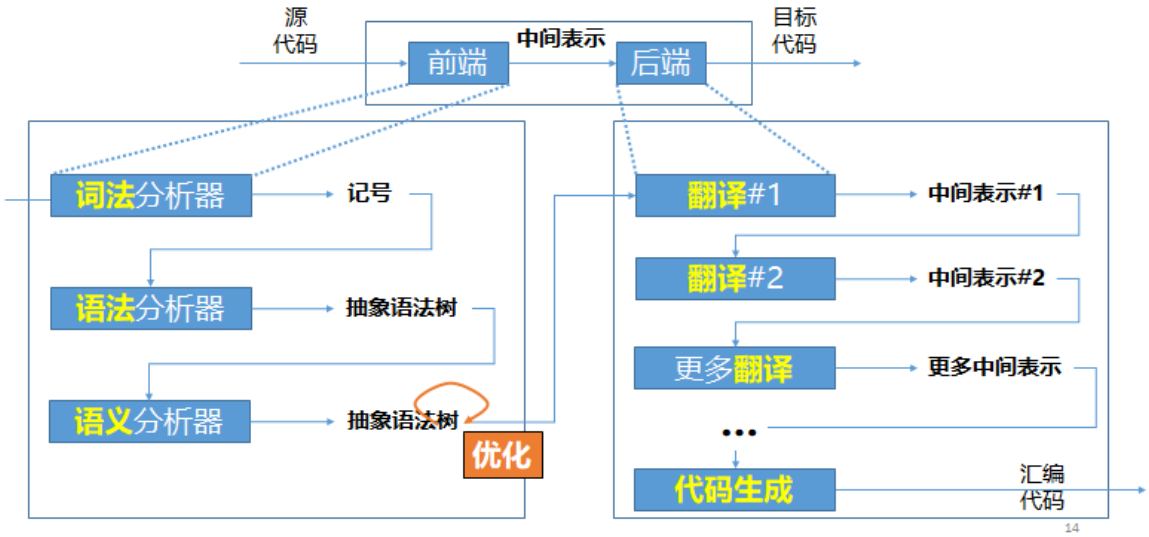
- 代码优化是对被优化的程序进行的一种**语义保持**的**变换**
- **语义保持**：
  - 程序的**可观察行为**不能改变
    - 可观察行为：Print，FILE，Network，。。。
- **变换**的目的是让程序能够比变换前：
  - 更小
  - 更快
  - 更节能
  - 。。。

前期优化小结

- 在抽象语法树上进行
- 典型优化方法
  - 常量折叠、代数优化、不可达代码删除等
- 特点
  - 简单
  - 可简化中后端处理
  - 优化能力有限
    - 有些优化很不方便，如不可达代码删除

- 前期优化

前期优化的位置



- 前期优化1：

常量折叠

- 观察
  - 程序中有些表达式的值**可在编译过程中计算得知**，无需等到运行阶段

```
a=3+5;
x=a+b;
printf("%d", x);
```

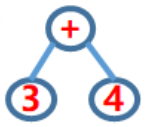
```
t1=true;
t2=false;
if (t1 && t2 && ...)
```

- 常量折叠基本思想
  - **在编译期计算表达式的值**
    - 例如：a=3+5 → a=8
    - 例如：if (true && false) → if (false)

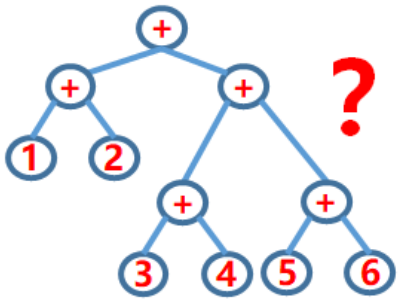
常量折叠算法

```
const_fold(Exp_t e) {
  while (e is still shrinking) {
    switch (e->kind)
    case EXP_ADD:
      Exp_t l = e->left;
      const_fold(l);
      Exp_t r = e->right;
      const_fold(e);
      if (l->kind==EXP_NUM && r->kind==EXP_NUM)
      e = Exp_Num_new (l->value + r->value);
      break;
    default:
      break;
  }
}
```

例：



抽象语法树#1



抽象语法树#2

常量折叠算法小结

- 容易实现、可以在语法树或者中间表示上进行
- 通常被实现成公共子函数被其它优化调用



- 必须要很小心遵守语言的语义
  - 例如：考虑溢出或异常
    - x = 4294967294 + 1

如果x是unsigned int:
x = **4294967295**

如果x是signed int:
x = **-1**

- 前期优化2：

代数化简

- 观察
  - 程序中有些表达式虽然包含变量（无法折叠成常数），但表达式的计算**可在编译过程中提前化简**

```
a = 0+b;
a = 1*b;
```

```
1024*a;
a+1024+b-1024;
```

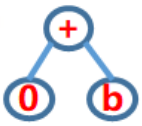
- 代数化简基本思想
  - **在编译期利用代数系统的性质对程序进行化简**
  - 例如：
    - a=0+b → a=b
    - a=1*b → a=b
    - 1024*a → a<<10
    - a+1024+b-1024 → a+b

19

代数化简算法

```
alg_simp(Exp_t e) {
  while (e is still shrinking) {
    switch (e->kind)
    case EXP_ADD:
      Exp_t l = e->left;
      alg_simp(l);
      Exp_t r = e->right;
      alg_simp(e);
      if (l->value==0 && r->kind==EXP_NUM)
      e = r;
      break;
    default:
      break;
  }
}
```

例：



抽象语法树#1



抽象语法树#2

20

代数化简算法小结

- 同样必须要很小心遵守语言的语义
  - 例如：考虑溢出或异常
    - (i-j)+(i-j) → (i+i)-(j-j)

假设i是signed short, j是signed int：

若i=16384 (0x4000); j=1 (0x1);
(i-j)+(i-j) = 32766 (0x7FFE)
a=(i+i); b=(j+j); a-b=-32770 (0xFFFF 7FFE)

- 前期优化3：

不可达代码删除

```
if (false)
  s1;
else
  s2;
```

- 观察
  - 程序中有些代码**永远不会执行到**
- 不可达代码删除基本思想
  - **在编译期移除程序中不会执行的代码**
  - 例如：

```
if (false)
  s1;
else
  s2;
```

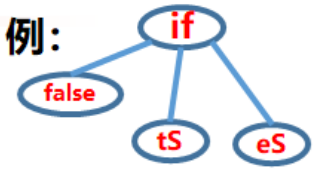
→

```
s2;
```

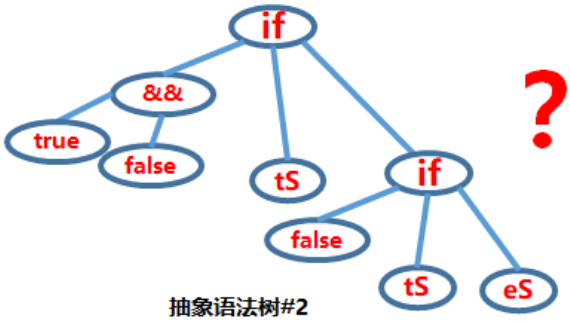
在控制流图上也可以进行这些优化，但在早期做这些优化可以简化中后端

不可达代码删除算法

```
deadcode(Stm_t s) {
  while(s is still shrinking) {
    switch (s->kind)
    case STM_IF:
      Exp_t e = s->condition;
      if (e->kind==EXP_FALSE)
      s = s->elseE;
      break;
  }
}
```



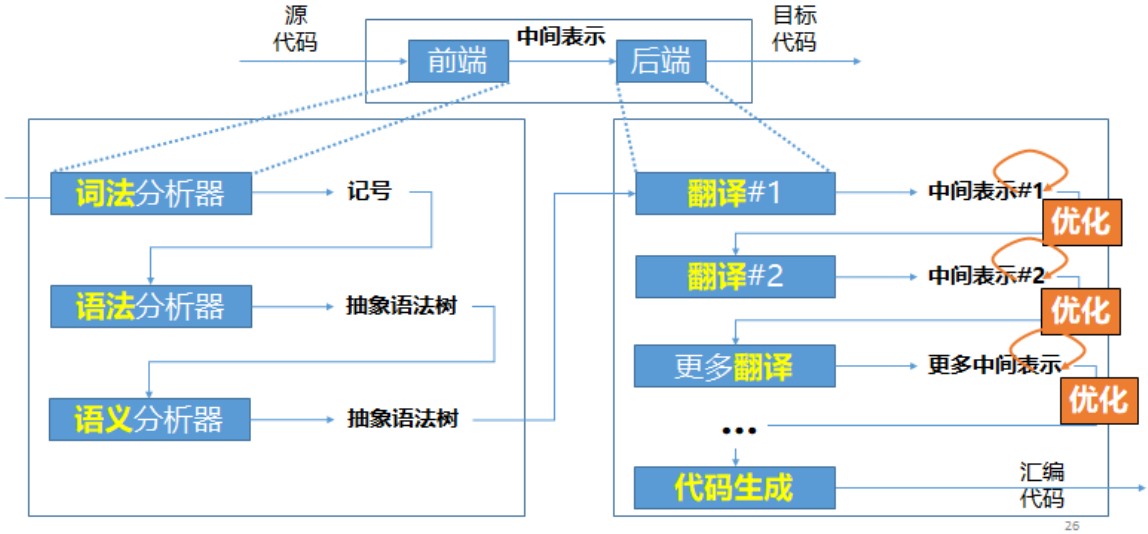
抽象语法树#1



抽象语法树#2

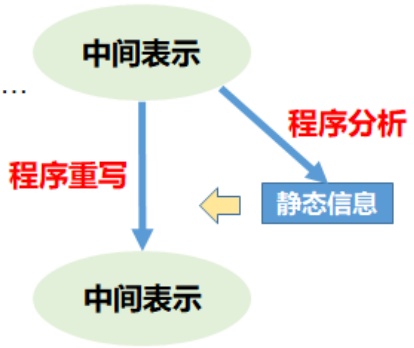
- 中期优化

中期优化的位置



中期优化的一般模式

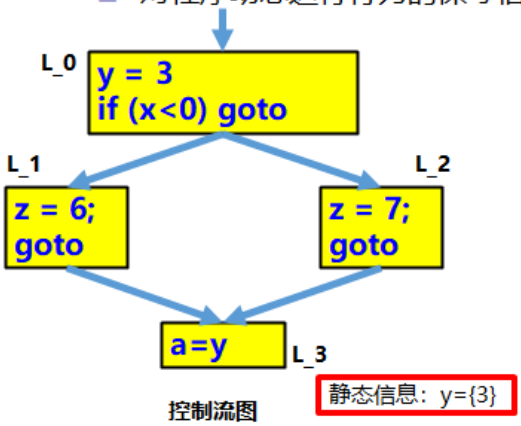
- **程序分析→程序重（优）写（化）**
  - 一般先进行程序分析，再基于程序分析的结果改写中间表示、进行优化
  - 优化通常是**全局**进行的
- 程序分析
  - 控制流分析，数据流分析，依赖分析，…
  - 得到被优化程序的**静态保守**信息
    - 对动态运行行为的保守估计
- 程序重写
  - 基于程序分析的信息对程序的重写

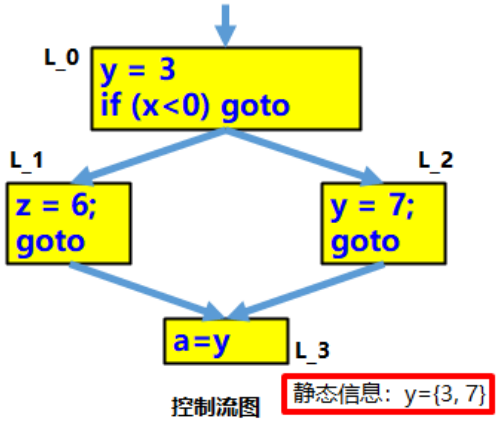


28

什么是静态保守信息？

- **静态保守**信息
  - 对程序动态运行行为的保守估计





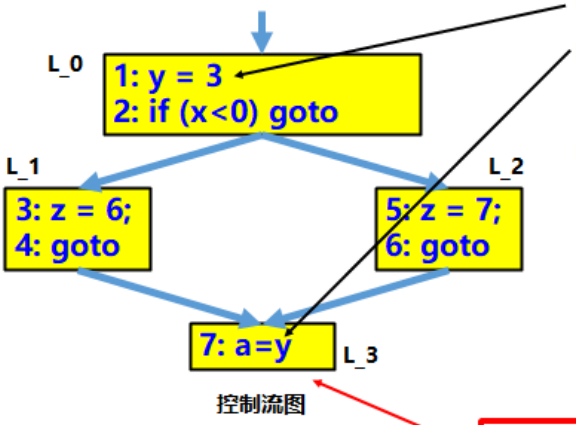
29

- 典型程序分析方法1：到达定义分析

它首先有defs，指的是变量a的所有被定义（即赋值）的语句。

之后生成gen kill，之后生成in out，通过ppt的算法。

到达定义分析



- **定义(def)**：对变量的赋值
- **使用(use)**：对变量值的读取
- **到达定义**（又叫到达定值）：对每个变量的使用点，有哪些定义可以到达？
  - 即：该变量的值是在哪儿赋值的？

问题：能把y替换成3么？
即：此处可能有哪些y的**定义**能够**到达**

如何进行到达定义分析？

- 对任何一条**定义**语句**[i: x = ...]**，给出两个集合：
  - **gen[i: x= ...] = {i}**
    - 经过这条语句，**新生成哪些定义**
  - **kill[i: x= ...] = defs[x]-{i}**
    - 经过这条语句，哪些之前的**定义不再往下传播**
    - defs[x]：在程序中出现的所有对于x的定义

i表示定义语句的行号

1: y = 3	gen[]={1}, defs[y]={1,4,5}, kill[]={4,5}
2: z = 4	gen[]={2}, defs[z]={2,6}, kill[]={6}
3: x = 5	gen[]={3}, defs[x]={3}, kill[]={}
4: y = 6	gen[]={4}, defs[y]={1,4,5}, kill[]={1,5}
5: y = 7	gen[]={5}, defs[y]={1,4,5}, kill[]={1,4}
6: z = 8	gen[]={6}, defs[z]={2,6}, kill[]={2}
7: a = y	gen[]={7}, defs[a]={7}, kill[]={}

如何进行到达定义分析？

- 对任何一条定义语句**[i: x = ...]**，给出两个集合：
  - **gen[i: x= ...] = {i}**
    - 经过这条语句，**新生成哪些定义**
  - **kill[i: x= ...] = defs[x]-{i}**
    - 经过这条语句，哪些之前的**定义不再往下传播**
    - defs[x]：在程序中出现的所有对于x的定义

i表示定义语句的行号

...

i-1: ...

↓ $out[s_{i-1}]$ = ↓ $in[s_i]$

i: x = ...

↓ $out[s_i] = gen[s_i] \cup (in[s_i]-kill[s_i])$

i+1: ...

...

旧的定义　旧的定义中不再往下传播的定义

新生成的定义　旧的继续往下传播的定义

- 到达定义计算：对于语句s_i（**只考虑基本块内**）
  - **$in[s_i] = out[s_{i-1}]$，$out[s_i] = gen[s_i] \cup (in[s_i]-kill[s_i])$**

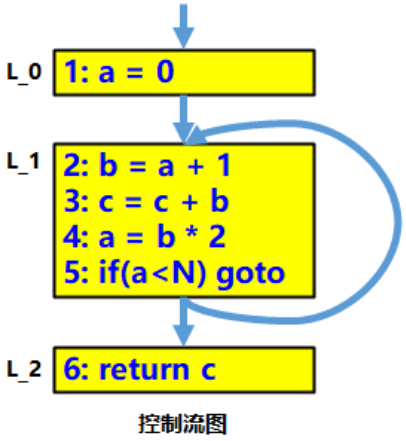
数据流方程

经过语句S_i前的到达定义　　经过语句S_i后的到达定义

到达定义分析示例

- 到达定义计算：对于的语句s（对于一般的控制流图）
  - in[s] = $\cup_{p \in pred(s)}$ out[p]，out[s] = gen[s]∪(in[s]-kill[s])

gen[d: x= ...] = {d}, kill[d: x= ...] = defs[x]-{d}



控制流图

语句	in/out	in/out	in/out	in/out
1	{} {}	{} {1}	{} {1}	
2	{} {}	{1} {1,2}	{1,2,3,4} {1,2,3,4}	
3	{} {}	{1,2} {1,2,3}	{1,2,3,4} {1,2,3,4}	
4	{} {}	{1,2,3} {2,3,4}	{1,2,3,4} {2,3,4}	
5	{} {}	{2,3,4} {2,3,4}	{2,3,4} {2,3,4}	
6	{} {}	{2,3,4} {2,3,4}	{2,3,4} {2,3,4}	

语句	1	2	3	4	5	6
gen[]	{1}	{2}	{3}	{4}	{}	{}
kill[]	{4}	{}	{}	{1}	{}	{}

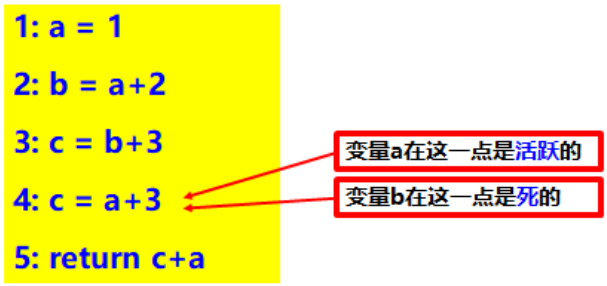
37

- 典型程序分析方法2：活性分析

这个方法是到达定义分析的后续，在得到了in out后，可以直接推出来在哪里是变量的活区间。

活性分析

- 活性分析的定义（又叫活跃变量分析）
  - 对某个变量x，最后一次使用是在哪个程序点?
  - 换言之
    - 如果变量x在程序点p(含)之后**被使用**，则x在p点是**活跃的**
    - 否则，如果变量x在程序点p(含)之后**未被使用**，则x在p点是**死的**

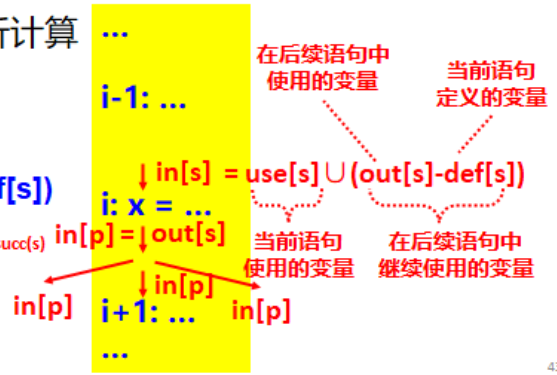


40

如何进行活性分析?

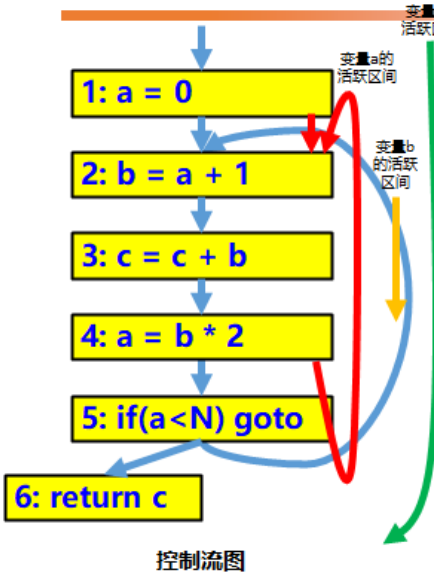
- 如果变量x在程序点p(含)之后**被使用**，则x在p点是**活跃的**
- 否则，如果变量x在程序点p(含)之后**未被使用**，则x在p点是**死的**

- 对任何一条语句 **[d: s]**，给出两个集合：
  - **use[d: s] = {x | 变量x在语句s中被使用}**
  - **def[d: s] = {x | 变量x在语句s中被定义}**
- **后向**数据流方程：活性分析计算
  - **out[s] = $\cup_{p \in succ(s)}$ in[p]**
    - 经过语句S后的活跃变量集合
  - **in[s] = use[s]∪(out[s]-def[s])**
    - 经过语句S前的活跃变量集合



活性分析**示例**

后向数据流方程：活性分析计算
out[s] = $\cup_{p \in succ(s)}$ in[p]
in[s] = use[s]∪(out[s]-def[s])



控制流图

语句的遍历顺序: 1, 2, 3, 4, 5, 6

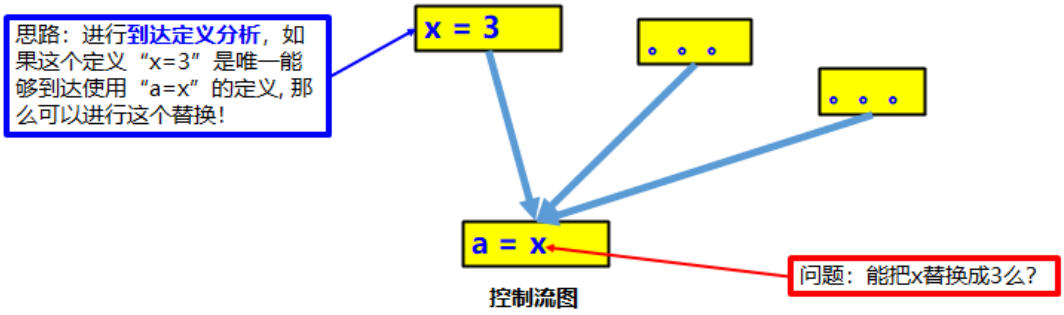
语句	in/out	in/out	in/out	in/out	in/out
1	{} {}	{} {}	{} {a}	{c} {a,c}	{c} {a,c}
2	{} {}	{a} {}	{a,c} {b,c}	{a,c} {b,c}	{a,c} {b,c}
3	{} {}	{b,c} {}	{b,c} {b}	{b,c} {b}	{b,c} {b,c}
4	{} {}	{b} {}	{b} {a}	{b,c} {a,c}	{b,c} {a,c}
5	{} {}	{a} {a}	{a,c} {a,c}	{a,c} {a,c}	{a,c} {a,c}
6	{} {}	{c} {}	{c} {}	{c} {}	{c} {}

语句	1	2	3	4	5	6
use[]	{}	{a}	{c,b}	{b}	{a}	{c}
def[]	{a}	{b}	{c}	{a}	{}	{}

- 典型优化方法1：常量传播

常量传播

- 常量传播
  - 将程序中使用的**变量替换为常量**



常量传播算法

```
const_prop(Prog_t p) {
    // 第一步：程序分析
    reaching_definition(p);
    // 第二步：程序改写
    foreach (stm s in p: y = x1, ..., xn) {
        foreach (use of xi in s) {
            if(the reaching def of xi is unique: xi = n)
                y = x1, ..., xi-1, n, xi+1, ..., xn
        }
    }
}
```


示例

// 示例程序
x = 1; {}
y = 2; {1}
z = x + y; {1,2}
a = z + 5; {1,2,3}
print (a); {1,2,3,4}

// 第一步：常量传播优化
x = 1;
y = 2;
z = 1 + 2;
a = z + 5 ;
print (a);

// 第二步：常量折叠优化
x = 1;
y = 2;
z = 3;
a = z + 5 ;
print (a);

// 第三步：常量传播优化
x = 1;
y = 2;
z = 3;
a = 3 + 5 ;
print (a);

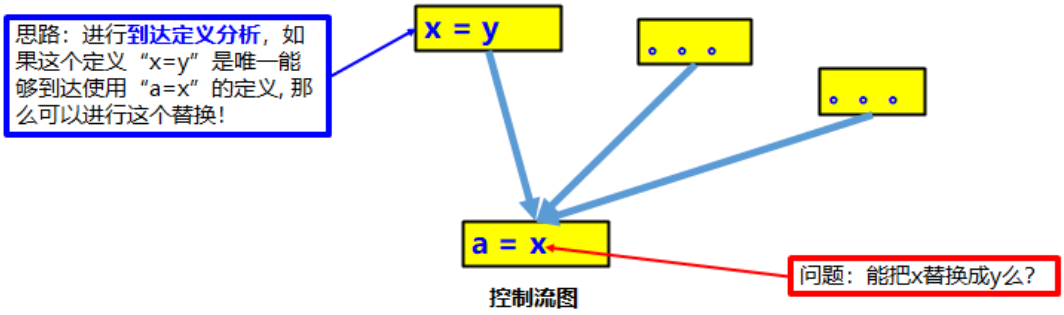
// 第四步：常量折叠优化
x = 1;
y = 2;
z = 3;
a = 8;
print (a);

// 第五步：常量传播优化
x = 1;
y = 2;
z = 3;
a = 8;
print (8);

- 典型优化方法2：拷贝传播

拷贝传播

- 拷贝传播
  - 将程序中使用的**变量替换为其他变量**



54

拷贝传播算法

```
copy_prop(Prog_t p) {
    // 第一步：程序分析
    reaching_definition(p);
    // 第二步：程序改写
    foreach (stm s in p: y = x1, ..., xn) {
        foreach (use of xi in s) {
            if(the reaching def of xi is unique: xi = z)
                y = x1, ..., xi-1, z, xi+1, ..., xn
        }
    }
}
```

55

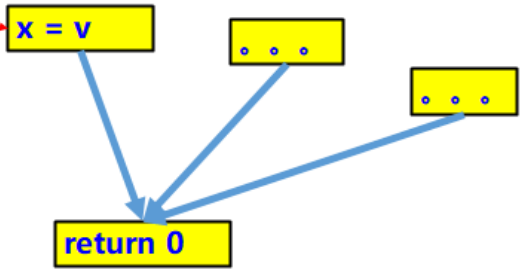
- 典型优化方法3：死代码删除

死代码删除 V.S. 不可达代码删除

- 死代码删除
  - 将程序中**无用的代码删除**

问题：能把这个语句删除吗？

思路：进行**活性分析**，如果x不属于该语句的活性out集，则可以将该语句移除。



控制流图

- 不可达代码删除
  - 观察
    - 程序中有些代码**永远不会执行到**
  - 不可达代码删除基本思想
    - **在编译期移除程序中不会执行的代码**

死代码删除算法

```
dead_code(Prog_t p) {
    // 第一步：程序分析
    live_variable_analysis(p);
    // 第二步：程序改写
    foreach (stm s in p: y = ...)
        if (y is NOT in live_out[s])
            remove (s);
}
```


示例

```
dead_code(Prog_t p) {
    // 第一步：程序分析
    live_variable_analysis(p);
    // 第二步：程序改写
    foreach (stm s in p: y = ...)
        if (y is NOT in live_out[s])
            remove (s);
}
```

// 第一步：常量传播优化
x = 1;
y = 2;
z = 1 + 2;
a = z + 5 ;
print (a);

// 第二步：死代码删除
~~x = 1;~~ {}
~~y = 2;~~ {}
~~z = 3;~~ {}
~~a = 8;~~ {} ;
print (8); {}

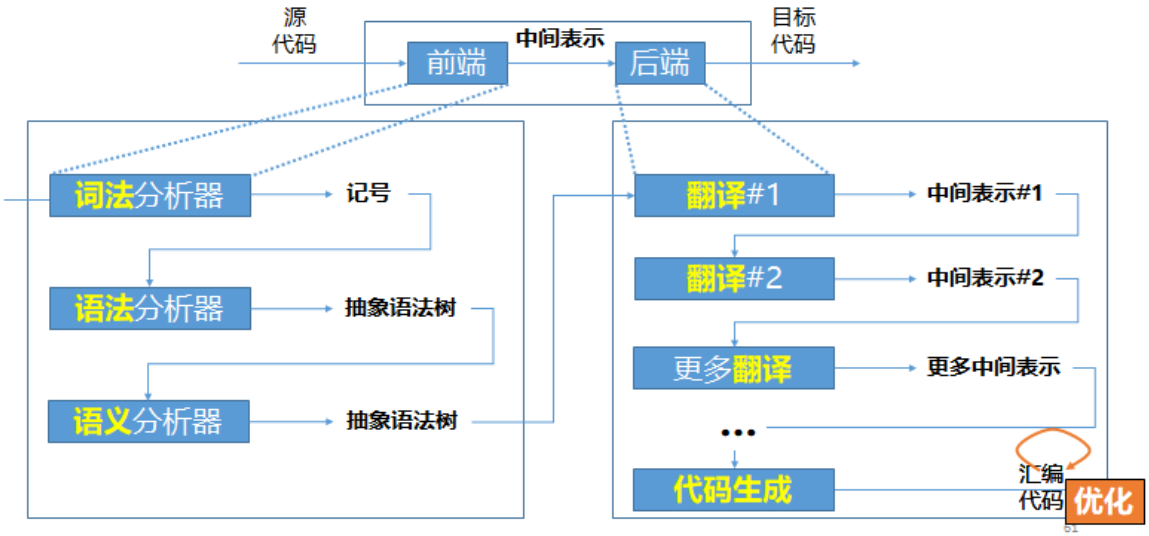
// 第三步：常量传播优化
x = 1;
y = 2;
z = 3;
a = 8;
print (8);

// 第四步：常量折叠优化
x = 1;
y = 2;
z = 3;
a = 8;
print (a);

// 第五步：常量传播优化
x = 1;
y = 2;
z = 3;
a = 8;
print (8);

- 后期优化

后期优化的位置



61

后期优化的特点

- 与目标机器相关
  - 寄存器数量、指令pipeline。。。
  - 需要对目标ISA足够了解
- 对程序的执行效率影响很大
  - 决定机器指令数量、执行速度、并行程度。。。

痛并快乐着！
（好玩、但是好难☺）

62

- 后期优化1：寄存器分配

寄存器分配

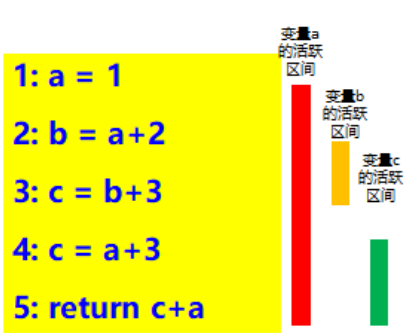
- 我们前面学习代码生成时假设目标机器有**无限多个寄存器**
  - 简化代码生成
- 真实目标机器只有**有限个寄存器**，如
  - RISC机器通常有32个寄存器，CISC机器通常有16个寄存器

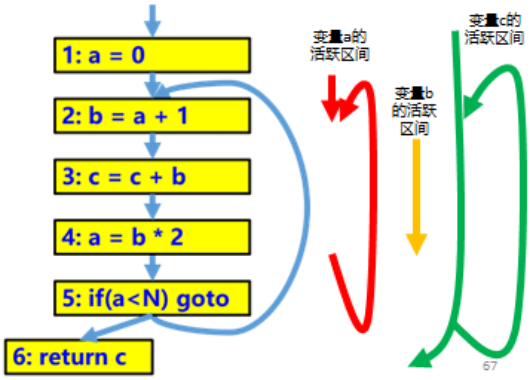
- **寄存器分配问题**
  - 重写中间代码，使其使用的寄存器数量不超过目标机器的寄存器数量
    - 将使用到的多个变量分配到少量物理寄存器中
    - 不改变源程序的行为

（这个问题，如果是线性的，是ACM的一个贪心题：离散化+模拟即可；但是这不一定是线性的，有分支循环等）

寄存器分配的思路

- 变量t1和t2可以使用**同一个寄存器**，当且仅当在程序的任何一个执行点上变量t1和t2中**至多有一个是活跃的**
  - 即：如果t1和t2**同时活跃**，则它们**不能使用同一个寄存器**





基于图着色的寄存器分配算法概览

- 第一步：计算活性分析
- 第二步：画出寄存器冲突图（RIG）
- 第三步：对RIG进行着色来分配寄存器
- 第四步：将无法分配寄存器的变量溢出到内存

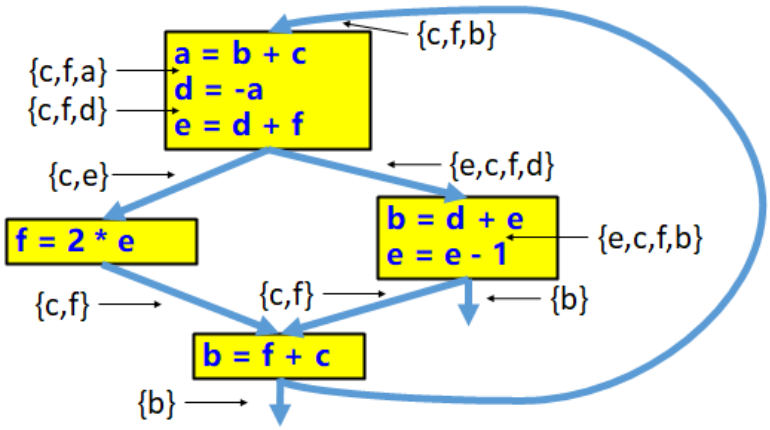
第一步：计算活性分析

后向数据流方程：活性分析计算

$$out[s] = \cup_{p \in succ(s)} in[p]$$

$$in[s] = use[s] \cup (out[s] - def[s])$$

- 计算程序每个执行点的**活跃变量**



示例

第二步：画出寄存器冲突图（RIG）

- 寄存器冲突图（Register Interference Graph，RIG）
  - 一个无向图
    - 每个**节点**是程序的一个**变量**
    - 如果程序中两个变量**同时活跃**，**添加一条边**连接这表示这两个变量的节点
  - 如果两个变量间**没有边连接**，则它们**可以使用同一个寄存器**
- 特点
  - 抽象出寄存器分配所需的所有信息
  - 构建RIG后，可以进行与目标机器无关的寄存器分配算法

第三步：对RIG进行着色来分配寄存器

- 图着色（Coloring）
  - 给图中**每个节点涂上一种颜色**，使得每条**边两端的节点颜色不同**
  - 如果一个图可以被着色成k种颜色，则称这个图为**k可着色的**
- 对RIG进行着色来分配寄存器
  - **颜色==寄存器**
    - 给每个节点涂颜色==给每个变量分配寄存器
  - 如果RIG是**k可着色的**，则该程序**可以使用不超过k个寄存器**
    - K==目标机器的寄存器数量

（显然这是一个指数级的，是NP-hard问题）

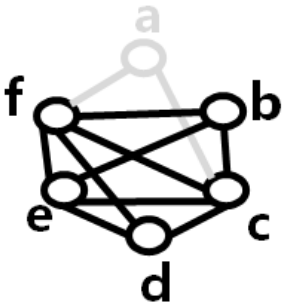
第三步：对RIG进行着色来分配寄存器

- 图着色的启发式算法
  - 对一个RIG进行k着色
- 算法步骤
  - 第一部分
    - 选择一个**邻居数量小于k**个的节点t，从RIG中**删除t和它所有的边**（暂假设总能找到）
    - 把**t放在栈顶**
    - 重复上述步骤只到RIG中**没有节点**
  - 第二部分
    - 从栈顶**弹出1个节点**
    - 给该节点**选择1种**与其邻居都**不同的颜色**
    - 重复上述步骤只到**栈为空**

第四步：将无法分配寄存器的变量溢出

- 如果所有节点都有大于等于k个邻居，怎么办？

考虑k=3

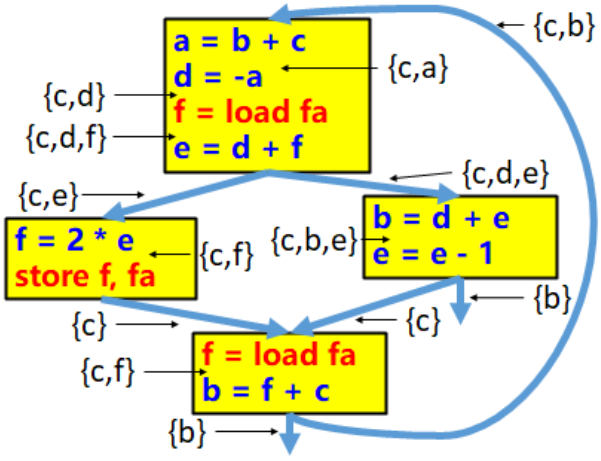


- **删除a之后就卡住了**

选择一个变量**溢出**！
（溢出的变量放在内存里）

第四步：将无法分配寄存器的变量溢出

- **重新计算活性分析**



- 乐观着色法：
  - 希望给溢出节点着色时其邻居用的颜色数量正好小于k
  - 如是，直接着色
- **溢出到内存**：
  - 如果乐观着色失败，将该节点溢出到内存
  - 在每次使用节点前插入load指令
  - 在每次定义节点后插入store指令

第四步：将无法分配寄存器的变量溢出

- 找到可行的图着色方案，可能需要**多次溢出**

- **溢出哪个变量**很关键，也很难
  - 可能的启发式算法
    - 选择与其他变量冲突最多的变量
    - 选择使用较少的变量
    - 避免溢出循环内部的变量
    - 。。。

结尾